

□> **Templates** : C:/Users/madan/Desktop/Supports de Cours 250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_2^{ème}_ann^{ée}_FR_01.pdf

Enseignant : BELACEL Madani

Établissement : École Normale Supérieure de Mostaganem

Module : Informatique

Niveau : ENS 2^{ème} ann^{ée}

Durée : 1h30

Année universitaire : 2025-2026

Fiche cours à Plateforme e-learning (Moodle)

Référentiel : benchmark 10 universités algériennes + pratiques USTO/Biskra/M'sila.

Public visé : Étudiants ENS 1^{ère} et 2^{ème} ann^{ée} à Départements Français et Anglais

Compétences pré-requis : Baccalauréat ; TIC niveau Licence 1 recommandé

Objectifs transversaux :

- Comprendre l'architecture machine et les systèmes d'exploitation
- Concevoir des algorithmes et structures de données algorithmiques
- Maîtriser les principes réseau et la sécurité informatique de base
- S'initier à Python pour l'enseignement

Mode d'évaluation : TD 40 % + TP Python 35 % + QCM 25 %

Bibliographie : Structure Machine 1 à Univ. Biskra ; Sécurité Informatique S6 à Univ. M'sila ; Goupille à Technologie des ordinateurs

Activité de mise en train (5 min)

Mini-exercice : Carte mentale de vos acquis ENS1

Prenez 3 minutes pour répondre individuellement à ces questions :

1. Citez trois types de données en Python (ex. int, str...).
2. Quelle est la différence entre une liste et un tuple ?
3. Écrivez la définition d'une fonction Python qui prend un nombre et retourne son carré.

Discussion en binôme (2 min) : Comparez vos réponses. Quelles difficultés avez-vous rencontrées ? Ces notions seront consolidées dans les séances à venir.

```
# Vérification rapide
def carre(x):
    return x ** 2

print(carre(5)) # 25
print(type(5)) # <class 'int'>
print(type(3.14)) # <class 'float'>
```

1) Objectifs pédagogiques

À l'issue de ce module, l'étudiant en ENS 2^{ème} année est capable de :

- Appliquer la **programmation orientée objet** (classes, héritage, encapsulation) en Python.
- Concevoir des **algorithmes avancés** (tri, recherche, récursivité) et estimer leur complexité.
- Modéliser des données avec le **modèle relationnel** et requêtes SQL de base.
- Comprendre l'architecture **Web** (HTML, CSS, JavaScript intro, HTTP).
- Utiliser **Git** et méthodes de travail collaboratif (branches, merge).
- Comparer l'**enseignement de l'informatique** au collège/lycée (Scratch, Python pédagogique).
- Articuler informatique, **stage** et **portfolio numérique** enseignant.

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** définit le cadre institutionnel des Ecoles Normales Supérieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algériennes** fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

2) Plan du cours

1. Rappel ENS1 et montée en compétence
2. Programmation orientée objet
3. Algorithmes avancés et récursivité
4. Bases de données et SQL
5. Développement Web fondamental
6. Git et ingénierie logicielle intro
7. Didactique de l'informatique à l'école
8. Projet ENS2 intégré

3) Contenu détaillé

timeline

title Parcours Module Informatique ENS 2^{ème} année

Séance 01 : Présentation du module : Objectifs et plan

Séances 02-03 : POO Python : Classes, héritage, polymorphisme

Séance 04 : Algorithmes avancés : Récursivité, tris, complexité

Séance 05 : Bases de données : Modèle relationnel, SQL
 Séance 06 : Développement Web : HTML, CSS, JavaScript
 Séance 07 : Git : Versions, branches, collaboration
 Séance 08 : Didactique : Enseigner l'informatique
 Séances 09-10 : Projet intégral : Application complète + soutenance

```
mindmap
  root((Module Informatique ENS2))
    Programmation
    POO Python
    Classes & Objets
    Héritage
    Algorithmique
    Récursivité
    Tris (bulle, fusion)
    Recherche dichotomique
    Données
    Modèle relationnel
    SQLite
    Requêtes SQL
    Web
    HTML5
    CSS3
    JavaScript
    Outils
    Git & GitHub
    Flask
    Debugage
    Pédagogie
    Didactique
    Scratch
    Fiche de séance
```

Révision ENS1 : les fondamentaux (10 min)

Avant d'aborder les nouveaux concepts, rappelons les acquis essentiels de l'ENS 1ère année :

Concept ENS1	Description	
Variables	Stockage de données typées	<
Structures conditionnelles	<code>if</code> , <code>elif</code> , <code>else</code>	<
Boucles	<code>for</code> , <code>while</code> pour itérer	<
Fonctions	Bloc de code réutilisable	<
Listes	Collection ordonnée et mutable	<
Tuples	Collection ordonnée et immuable	<
Dictionnaires	Paires clé-valeur	<

```
# Exemple de révision : calcul de moyenne avec une fonction
def moyenne(notes):
    """Calcule la moyenne d'une liste de notes."""
    if not notes:
        return 0
    return sum(notes) / len(notes)

# Test
notes_eleve = [12, 15, 18, 10]
print(f"Moyenne : {moyenne(notes_eleve):.2f}") # Moyenne : 13.75
```

Transition ENS1 à ENS2 : En 1^{ère} année, vous avez appris à écrire des scripts isolés. En 2^{ème} année, nous passons à la **programmation structurée** : projets multi-fichiers, classes, bases de données, travail collaboratif. Chaque nouveau concept sera relié à vos acquis antérieurs.

Chapitre 1 – Bilan et enjeux ENS2 (10 min)

Bilan ENS1 : Vous avez acquis les bases de l'algorithmique (variables, boucles, fonctions), de l'architecture machine et de la programmation Python. Vous êtes capables d'écrire un script de 50-100 lignes résolvant un problème simple.

Passage au niveau supérieur : ENS2 marque le passage du script isolé au **projet logiciel structuré**. Vous allez apprendre à organiser votre code en classes (POO) pour des programmes plus complexes, manipuler des données volumineuses avec SQL, collaborer avec Git comme dans une équipe de développement, et préparer votre future pratique d'enseignant en informatique.

Compétences futures de l'enseignant : L'enseignant d'informatique du XXI^e siècle doit maîtriser l'algorithmique, la programmation, la culture numérique, et les enjeux éthiques de l'IA. Ce module construit ces compétences de manière intégrée.

Chapitre 2 – POO (20 min)

Concepts : Classe, objet, attribut, méthode. Constructeur `__init__`. Encapsulation, héritage simple.

Exemple concret – contexte scolaire :

```
class Eleve:
    def __init__(self, nom, prenom, niveau):
        self.nom = nom
        self.prenom = prenom
        self.niveau = niveau
        self.notes = []

    def ajouter_note(self, note, matiere):
        self.notes.append({"matiere": matiere, "note": note})

class Enseignant(Eleve):
    def __init__(self, nom, prenom, matiere):
        super().__init__(nom, prenom, "Enseignant")
        self.matiere_principale = matiere
```

Lien avec l'enseignement : En collège, la POO peut être introduite via Scratch (les lutins sont des objets avec attributs et scripts). Ce parallèle sera développé en séance 7 (Didactique).

Mini-exercice : Identifiez 3 objets du monde scolaire et listez leurs attributs et méthodes (ex. : SalleDeClasse à attributs : numero, capacite, etage ; méthodes : afficher_info(), est_disponible()).

Chapitre 3 – Algorithmes avancés (15 min)

Algorithmes abordés :

- **Tri bulle** : comparaisons successives d'éléments adjacents – simple mais lent ($O(n^2)$)
- **Tri fusion** : algorithme récursif divisant le tableau en deux – efficace ($O(n \log n)$)
- **Recherche dichotomique** : recherche dans un tableau trié en éliminant la moitié à chaque étape ($O(\log n)$)
- **Récursivité** : factorielle, Fibonacci – une fonction qui s'appelle elle-même

Exemple concret : tri des notes d'une classe

```
notes = [12, 5, 18, 8, 15, 20, 10]
notes_triees = sorted(notes)
print(f"Notes tri  es : {notes_triees}")
```

Lien avec l'enseignement secondaire : Au coll  ge, les   l  ves comparent des nombres sans strat  gie explicite    l'algorithme de tri formalise cette intuition. En NSI, les algorithmes de tri sont au programme de Premi  re et Terminale.

Chapitre 4    Bases de donn  es (15 min)

Concepts fondamentaux :

- **Mod  le relationnel** : organisation des donn  es en tables reli  es par des cl  s
- **SQL** : langage de requ  tes structur  es
- **Int  grit  ** : contraintes (cl   primaire, cl     trang  re, types)
- **Normalisation** : 1  re forme normale (  viter les r  p  titions)

Exemple    gestion des notes d'une   cole :

```
CREATE TABLE Eleve (
  id_eleve INTEGER PRIMARY KEY,
  nom TEXT NOT NULL,
  prenom TEXT NOT NULL,
  classe TEXT
);

CREATE TABLE Note (
  id_note INTEGER PRIMARY KEY,
  valeur REAL CHECK(valeur >= 0 AND valeur <= 20),
  matiere TEXT,
  id_eleve INTEGER,
  FOREIGN KEY (id_eleve) REFERENCES Eleve(id_eleve)
);
```

Lien avec l'enseignement : Le tableur (Excel, LibreOffice Calc) est une introduction concr  te aux bases de donn  es    chaque ligne est un enregistrement, chaque colonne un champ. La transition tableur    SQL est naturelle en SNT (Th  me "Donn  es structur  es").

Chapitre 5    Web (15 min)

Architecture :

- **Client-serveur** : le navigateur (client) envoie des requ  tes HTTP au serveur qui r  pond
- **Frontend** : HTML (structure), CSS (style), JavaScript (interactivit  )
- **Backend** : Flask (Python) pour cr  er des applications Web simples
- **API REST** : interface standard pour   changer des donn  es entre applications

Exemple    carnet de notes en ligne :

```
<!DOCTYPE html>
<html>
<head><title>Carnet de notes</title></head>
<body>
<h1>Notes de la classe ENS2</h1>
<table>
<tr><th>Nom</th><th>Note</th></tr>
<tr><td>Ali</td><td>15</td></tr>
<tr><td>Sara</td><td>18</td></tr>
</table>
```

```
</body>
</html>
```

Accessibilité Web : Utilisez des balises sémantiques (<header>, <main>, <nav>), des textes alternatifs pour les images (alt), un contraste suffisant. Ces bonnes pratiques sont essentielles pour l'enseignement inclusif.

Chapitre 6 – Git et qualité (10 min)

Commandes essentielles : git init, git add, git commit, git branch, git merge

Qualité logicielle : Documentation (README, docstrings), tests unitaires (assert), .gitignore

```
# Exemple de test unitaire
def moyenne(notes):
    return sum(notes) / len(notes) if notes else 0

assert moyenne([10, 20]) == 15.0
assert moyenne([]) == 0
print("Tous les tests passent à")
```

Lien avec l'enseignement : Git est un excellent outil pour enseigner la collaboration : les branches sont comme des groupes d'élèves qui travaillent sur des parties différentes d'un projet, puis fusionnent leurs travaux.

Chapitre 7 – Didactique informatique (10 min)

Contenus abordés :

- **Programmes officiels** : SNT Seconde (1h30/sem), NSI Première/Terminale (4h/sem)
- **Démarche projet** : les élèves construisent un projet sur plusieurs séances
- **Scratch à Python** : transition de la programmation visuelle au code textuel
- **Différenciation** : adapter les exercices selon le niveau des élèves
- **Évaluation par compétences** : grilles critériées, auto-évaluation

Charte usage IA en classe : L'IA générative (ChatGPT, etc.) peut être un outil d'apprentissage si son usage est encadré. Propositions : demander aux élèves d'analyser/critiquer le code généré par IA plutôt que de le recopier.

Exemple de fiche de séance type :

Phase	Durée	Activité
Mise en train	5 min	Question rapide en binôme
Déroulé	15 min	Démonstration par l'enseignant
Pratique guidée	15 min	Exercice en autonomie
Bilan	5 min	Synthèse collective

Chapitre 8 – Projet ENS2 (5 min)

Livrable attendu : Application Web à gère ou module Python POO + base SQLite + dépôt Git + fiche séance pédagogique 45 min pour cycle collège.

Exemples de sujets :

1. **Carnet de notes interactif** – classes Python + base SQLite + interface Flask

2. **Quiz pédagogique** : POO (Question, Quiz) + SQL (banque de questions) + Web (interface)

3. **Gestion de bibliothèque scolaire** : classes (Livre, Adherent, Emprunt) + SQL + Git

Calendrier prévisionnel :

- Semaines 2-4 : conception POO et prototypage
- Semaines 5-6 : intégration base de données
- Semaines 7-8 : interface Web et finalisation
- Semaine 9 : rédaction fiche pédagogique
- Semaine 10 : soutenance et démonstration

Cas d'étude intégrateur : Gestion d'une bibliothèque scolaire

Ce cas relie tous les chapitres du module et servira de fil conducteur.

Concept	Application au cas Bibliothèque
POO (Ch.2)	Classes <code>Livre</code> , <code>Adherent</code> , <code>Emprunt</code>
Algorithmes (Ch.3)	Recherche dichotomique d'un livre par titre, tri des emprunts par date
BDD (Ch.4)	Schéma relationnel : tables Livre, Adherent, Emprunt avec clés étrangères
Web (Ch.5)	Interface HTML pour rechercher/emprunter des livres
Git (Ch.6)	Développement partagé avec branches par fonctionnalité
Didactique (Ch.7)	Fiche séance pour enseigner la POO via ce projet en collège
Projet (Ch.8)	Application complète livrée avec documentation et tests

Synthèse transversale

```

graph TD
  A[Révision ENS1] --> B[POO]
  A --> C[Algorithmes]
  B --> D[Base de données]
  C --> D
  D --> E[Web]
  E --> F[Git]
  B --> G[Didactique]
  C --> G
  D --> G
  E --> G
  F --> G
  G --> H[Projet ENS2]
  B --> H
  D --> H
  E --> H
  F --> H

```

Questions de révision transversale :

1. Comment modéliserez-vous un système de gestion des notes avec les concepts POO ?
2. Quel algorithme de tri utiliserez-vous pour classer 100 000 élèves par moyenne ? Justifiez.
3. Comment relierez-vous une table `Eleve` et une table `Note` en SQL ?

4. Quels sont les avantages de Git pour un projet pédagogique en groupe ?
5. Comment adapteriez-vous le contenu de ce module pour une classe de 4^{ème} au collège ?

Repères historiques en informatique

Année	Événement	Lien avec le cours
1967	Simula est premier langage POO	Origine des classes et objets (Ch.2)
1989	Tim Berners-Lee invente le Web	Naissance d'HTML (Ch.5)
1991	Guido van Rossum crée Python	Le langage utilisé dans ce module
2005	Création de Git par Linus Torvalds	Gestion de versions (Ch.6)
2012	Scratch 2.0 en ligne	Programmation visuelle pour l'éducation (Ch.7)

4) Modalités d'évaluation

Composante	Poids
TP POO + SQL	35 %
Projet intégré (code + Git)	40 %
Fiche séance pédagogique	25 %

5) Bibliographie

- Gutttag est *Introduction to Computation and Programming Using Python*.
- Connolly & Begg est *Database Systems* (chapitres introductifs).
- MDN Web Docs est HTML, CSS, JavaScript.
- Eduscol est Enseignement de l'informatique au lycée.
- Downey est *Think Python* (disponible gratuitement en ligne).
- Chacon & Straub est *Pro Git* (livre gratuit en ligne).

QCM d'auto-évaluation

1. Quel est le poids du projet intégré dans l'évaluation finale ?
a) 25 % b) 40 % c) 35 %
2. Quelle séance introduit la programmation orientée objet ?
a) Séance 01 b) Séance 02 c) Séance 04
3. Le module NSI au lycée a un volume horaire de :
a) 1h30/semaine b) 4h/semaine c) 2h/semaine
4. Quel outil de versionnement est enseigné dans ce module ?
a) SVN b) Git c) Mercurial

5. Quel langage est utilisé pour la transition pédagogique Scratch à texte ?

a) Java b) Python c) C++

6. Quel est l'ordre de complexité de la recherche dichotomique ?

a) $O(n)$ b) $O(\log n)$ c) $O(n^2)$

7. En SQL, quelle clause permet de filtrer les résultats ?

a) ORDER BY b) WHERE c) JOIN

8. Quelle commande Git crée une nouvelle branche ?

a) `git branch <nom>` b) `git checkout <nom>` c) `git merge <nom>`

9. Quel langage est utilisé pour structurer le contenu d'une page Web ?

a) CSS b) JavaScript c) HTML

10. Quelle stratégie pédagogique consiste à passer de Scratch au code textuel ?

a) L'apprentissage par projet b) La transition visuel à texte c) La classe inversée

Conclusion

Ce module vous accompagne de la maîtrise technique (POO, algorithmes, bases de données, Web, Git) jusqu'à la transposition didactique pour l'enseignement secondaire. Chaque séance construit une brique du projet intégral qui viendra consolider l'ensemble des compétences visées.

Prochain cours : Programmation orientée objet : classes, objets et méthodes en Python.

Note pédagogique ENS

Transposition didactique : La vue d'ensemble du module peut être présentée aux élèves de collège/lycée sous forme de carte mentale. Dans la classe de 3^{ème}, on peut illustrer le lien entre les blocs Scratch (programmation visuelle) et le code Python (programmation textuelle) pour montrer la progressivité des apprentissages. En SNT Seconde, le thème "Internet" peut être relié à la séance Web ; le thème "Données structurées" à la séance SQL.

Conseil pour l'enseignant formateur : Utilisez ce plan de cours comme outil de communication avec vos étudiants. Distribuez-le en début de module pour qu'ils visualisent la cohérence d'ensemble. Proposez-leur de cocher les séances au fur et à mesure, ce qui renforce le sentiment de progression. N'hésitez pas à revenir sur ce plan à chaque début de séance pour contextualiser.

Section de révision : En fin de module, organisez une séance de révision collaborative. Chaque étudiant prépare une fiche de synthèse sur un chapitre, puis les groupes échangent leurs fiches. Cette activité, appelée "classe puzzle" (Jigsaw classroom), renforce l'apprentissage actif et la responsabilisation.

Prolongement : Les étudiants peuvent créer une carte mentale numérique (XMind, FreeMind) de leur propre progression dans le module, en y ajoutant des ressources personnelles (liens, notes, exercices) : cela deviendra une base pour leur portfolio enseignant.

Templates : C:/Users/madan/Desktop/Supports de Cours 250626/Template/Pg-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_2^{ème}_année_FR_02.pdf

Cours Informatique à l'ENS 2^{ème} année à l'École Supérieure 02 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'ENS 2^{ème} année à l'École Supérieure de France

Durée : 1h30

Année universitaire : 2025-2026

Fiche cours à l'Plateforme e-learning (Moodle)

Résumé : benchmark 10 universités algériennes + pratiques USTO/Biskra/M'sila.

Public visé : Étudiants ENS 2^{ème} année à l'École Supérieure de France

Compétences pré-requis : Python bases (ENS1 séance 09), algorithmique (ENS1 séance 05)

Objectifs transversaux :

- Appliquer la programmation orientée objet (classes, héritage, encapsulation) en Python
- Concevoir des algorithmes avancés (tri, recherche, récursivité) et estimer leur complexité
- Modéliser des données avec le modèle relationnel et requêtes SQL de base
- Comprendre l'architecture Web (HTML, CSS, JavaScript intro, HTTP)
- Utiliser Git et méthodes de travail collaboratif (branches, merge)
- Comparer l'enseignement de l'informatique au collège/lycée (Scratch, Python pédagogique)

Mode d'évaluation : TP POO + SQL 35 % à l'Projet intégrée 40 % à l'Fiche séance pédagogique 25 %

Bibliographie : Guttag à l'*Introduction to Computation and Programming Using Python* ; Lutz à l'*Learning Python*

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Définir une classe Python avec attributs et méthodes
- Instancier des objets
- Utiliser le constructeur `__init__`
- Comprendre la différence entre classe et instance
- Créer une classe simple adaptée à un contexte scolaire

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- Arrêté ministériel portant création des ENS (cycle Licence/Master) à l'définit le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.

- **Programme national d'Informatique des ENS algériennes** fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Paradigme procédural vs orienté objet
2. Classe et objet : concepts fondamentaux
3. Attributs et méthodes
4. Constructeur `__init__`
5. Variables de classe vs d'instance
6. Exemple : classe `Eleve`
7. Exemple : classe `Evaluation`

1. Pourquoi la POO ? (10 min)

Programmation procédurale (ENS1) : fonctions + données séparées.

Programmation orientée objet : regroupe données et comportement dans des **objets**.

Avantages :

- Organisation naturelle (on pense en termes d'objets du monde réel)
- Réutilisable (une classe = un moule à objets)
- Maintenable (modifier une classe n'affecte pas le reste)
- Extensible (héritage)

Exemple concret : dans la vie scolaire, on manipule des élèves, des Enseignants, des Notes, des Classes etc ce sont des objets.

```

flowchart LR
    A[Problème réel<br/>gestion scolaire] --> B[Identifier les objets<br/>élève, Note, Classe]
    B --> C[Définir les classes<br/>attributs + méthodes]
    C --> D[Instancier des objets]
    D --> E{Objet cohérent ?}
    E -->|Oui| F[Réutiliser<br/>dans d'autres contextes]
    E -->|Non| B
    F --> G[Maintenance<br/>et évolution]
  
```

2. Classe et objet (15 min)

Classe : modèle, plan de construction.

Objet : réalisation concrète du modèle (instance).

```
# Définition de la classe
class Eleve:
    pass

# Création d'objets (instances)
eleve1 = Eleve()
eleve2 = Eleve()

print(type(eleve1)) # <class '__main__.Eleve'>
```

La classe est le moule ; l'objet est l'élément produit par ce moule.

```
classDiagram
class Eleve {
+string ecole
+int total_eleves
+string nom
+int age
+__init__(nom, age)
+se_presenter()
}
class EleveAvance {
+list notes
+string groupe
+ajouter_note(note)
+moyenne()
+afficher()
}
Eleve <|-- EleveAvance
note for Eleve "Classe = plan de construction"
note for EleveAvance "Objet = instance concr te"
```

3. Attributs et m thodes (15 min)

Attribut : variable attach e   un objet (son  tat).

M thode : fonction attach e   un objet (son comportement).

```
class Eleve:
    def __init__(self, nom, age):
        self.nom = nom # Attribut d'instance
        self.age = age

    def se_presenter(self): # M thode
        print(f"Je m'appelle {self.nom} et j'ai {self.age} ans.")

# Utilisation
e = Eleve("Ali", 20)
print(e.nom) # Acc s   l'attribut
e.se_presenter() # Appel de la m thode
```

Noter : `self` repr sente l'instance courante   c'est le premier param tre de toute m thode.

Mini-exercice : Sans regarder la suite, ajoutez une m thode `est_majeur()`   la classe `Eleve` qui retourne `True` si l' ge est `>= 18` et `False` sinon. Testez-la avec `e = Eleve("Ali", 20)`.

```
# Solution
class Eleve:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

    def se_presenter(self):
        print(f"Je m'appelle {self.nom} et j'ai {self.age} ans.")
```

```
def est_majeur(self):
    return self.age >= 18

# Test
e = Eleve("Ali", 20)
print(e.est_majeur()) # True
print(Eleve("Sara", 16).est_majeur()) # False
```

Cas concret : Cette méthode `est_majeur()` correspond à une vérification que fait un enseignant pour savoir quels élèves peuvent participer à une sortie scolaire sans autorisation parentale supplémentaire.

4. Constructeur `__init__` (15 min)

La méthode `__init__` est appelée automatiquement à la création de l'objet.

```
class Evaluation:
    def __init__(self, matiere, note, coefficient=1):
        self.matiere = matiere
        self.note = note
        self.coefficient = coefficient

    def ponderer(self):
        return self.note * self.coefficient

# Création
eval1 = Evaluation("Maths", 15, 2)
eval2 = Evaluation("Français", 12)
print(eval1.ponderer()) # 30
print(eval2.ponderer()) # 12 (coefficient par défaut = 1)
```

5. Variables de classe (10 min)

Attributs partagés par toutes les instances.

```
class Eleve:
    ecole = "ENS Mostaganem" # Variable de classe

    def __init__(self, nom):
        self.nom = nom # Variable d'instance

e1 = Eleve("Ali")
e2 = Eleve("Sara")
print(e1.ecole) # ENS Mostaganem
print(e2.ecole) # ENS Mostaganem
```

Utilité : constantes, compteurs, configuration partagée.

5bis. La méthode `__str__` pour l'affichage (5 min)

La méthode `__str__` définit comment un objet est affiché avec `print()`.

```
class Eleve:
    def __init__(self, nom, prenom):
        self.nom = nom
        self.prenom = prenom

    def __str__(self):
```

```

return f"ÃlÃve : {self.prenom} {self.nom}"

e = Eleve("Belacel", "Ali")
print(e) # ÃlÃve : Ali Belacel
print(str(e)) # Idem

```

Utilit   pédagogique : En classe, `__str__` permet d'afficher des objets de mani  re lisible    on   vite le `<__main__.Eleve object at 0x...>` incompr  hensible.

6. Exemple complet : classe Eleve (20 min)

```

class Eleve:
    ecole = "ENS Mostaganem"
    total_eleves = 0 # Compteur partag  

    def __init__(self, nom, prenom, groupe):
        self.nom = nom
        self.prenom = prenom
        self.groupe = groupe
        self.notes = []
        Eleve.total_eleves += 1

    def ajouter_note(self, note):
        if 0 <= note <= 20:
            self.notes.append(note)
        else:
            print("Note invalide")

    def moyenne(self):
        if self.notes:
            return sum(self.notes) / len(self.notes)
        return 0.0

    def afficher(self):
        print(f"{self.prenom} {self.nom}    {self.groupe}")
        print(f"Notes : {self.notes}")
        print(f"Moyenne : {self.moyenne():.2f}")

# Test
e = Eleve("Belacel", "Ali", "ENS1-A")
e.ajouter_note(15)
e.ajouter_note(12)
e.ajouter_note(18)
e.afficher()
print(f"Total ÃlÃves : {Eleve.total_eleves}")

```

6bis. Encapsulation et propri  t  s (15 min)

Principe : prot  ger les attributs en contr  lant leur acc  s avec des propri  t  s.

```

class Note:
    def __init__(self, valeur):
        self._valeur = 0 # Attribut "prot  g  " (convention : _)
        self.valeur = valeur # Passe par le setter

    @property
    def valeur(self):
        return self._valeur

    @valeur.setter
    def valeur(self, v):

```

```

if 0 <= v <= 20:
    self._valeur = v
else:
    raise ValueError("La note doit  tre entre 0 et 20")

# Test
n = Note(15)
print(n.valeur) # 15 (getter)
n.valeur = 18 # Setter
# n.valeur = 25 # D clenche ValueError

```

En contexte scolaire : L'encapsulation est comme un carnet de notes tenu par l'enseignant : on ne peut pas modifier une note sans respecter les r gles (entre 0 et 20). Le d corateur `@property` est le "guichet" qui v rifie chaque modification.

7. Exercice guid  : classe `Matiere` (10 min)

Cr er une classe `Matiere` avec :

- Attributs : `nom`, `coefficient`, `nb_heures`
- M thode `__init__` avec param tres
- M thode `afficher()` qui affiche les informations
- M thode `est_importante()` qui retourne `True` si `coefficient > 2`

```

class Matiere:
    def __init__(self, nom, coefficient, nb_heures):
        self.nom = nom
        self.coefficient = coefficient
        self.nb_heures = nb_heures

    def afficher(self):
        print(f"{self.nom} (coef {self.coefficient}, {self.nb_heures}h)")

    def est_importante(self):
        return self.coefficient > 2

```

7bis. Cas d' tude : Syst me de gestion de notes (15 min)

Assemblons les concepts dans un exemple r aliste multi-classes :

```

class Matiere:
    def __init__(self, nom, coefficient=1):
        self.nom = nom
        self.coefficient = coefficient

class Note:
    def __init__(self, valeur, matiere):
        self.valeur = valeur
        self.matiere = matiere

class Eleve:
    ecole = "ENS Mostaganem"

    def __init__(self, nom, prenom):
        self.nom = nom
        self.prenom = prenom
        self.notes = []

```

```
def ajouter_note(self, valeur, matiere):
    note = Note(valeur, matiere)
    self.notes.append(note)

def moyenne_generale(self):
    if not self.notes:
        return 0.0
    total_pondere = sum(n.valeur * n.matiere.coefficient for n in self.notes)
    total_coef = sum(n.matiere.coefficient for n in self.notes)
    return total_pondere / total_coef if total_coef else 0.0

def __str__(self):
    return f"{self.prenom} {self.nom} â€” Moy : {self.moyenne_generale():.2f}"

# Utilisation
maths = Matiere("Maths", 3)
francais = Matiere("FranÃ§ais", 2)

ali = Eleve("Belacel", "Ali")
ali.ajouter_note(15, maths)
ali.ajouter_note(12, francais)

sara = Eleve("Benali", "Sara")
sara.ajouter_note(18, maths)
sara.ajouter_note(14, francais)

print(ali) # Ali Belacel â€” Moy : 13.80
print(sara) # Sara Benali â€” Moy : 16.40
```

Discussion : Cet exemple illustre la collaboration entre classes (Eleve, Note, Matiere). Chaque classe a une **responsabilité unique** (Single Responsibility Principle), ce qui rend le code maintenable et évolutif.

```
sequenceDiagram
    participant User as Utilisateur
    participant Eleve
    participant Note
    participant Matiere

    User->>Eleve: creer_eleve("Ali")
    User->>Matiere: creer_matiere("Maths", 3)
    User->>Eleve: ajouter_note(15, maths)
    Eleve->>Note: creer Note(15, maths)
    Note-->>Eleve: note cr  e
    User->>Eleve: moyenne_generale()
    Eleve->>Note: acc  s n.valeur, n.matiere.coefficient
    Note-->>Eleve: donn  es
    Eleve-->>User: r  sultat 15.0
```

QCM d'auto-évaluation

1. Qu'est-ce qu'une classe en POO ?

a) Une variable globale b) Un plan de construction pour créer des objets " c) Une fonction spéciale

2. Quel mot-clé représente l'instance courante dans une méthode Python ?

a) this b) self c) instance

3. La méthode `init` est :

a) Appelé manuellement par le programmeur b) Appelé automatiquement à la création de l'objet c) Optionnelle pour toute classe

4. Quelle est la différence entre variable de classe et variable d'instance ?

a) La variable de classe est partagée par toutes les instances b) La variable d'instance est partagée c) Il n'y a pas de différence

5. Que retourne `Eleve.total_eleves` dans l'exemple du cours ?

a) Le nombre de méthodes de la classe b) Le nombre total d'instances créées c) Le nom de l'cole

6. Quelle méthode spéciale permet d'afficher un objet avec `print()` ?

a) `__init__` b) `__str__` c) `__repr__`

7. À quoi sert le décorateur `@property` en Python ?

a) À définir une variable de classe b) À créer un getter avec contrôle c) À initialiser un objet

8. Quel nom résume le principe qu'une classe ait une responsabilité unique ?

a) DRY b) SRP (Single Responsibility Principle) c) KISS

8. Conclusion (5 min)

La POO organise le code autour d'objets qui encapsulent données et comportement. Les classes sont réutilisables et faciles à maintenir.

Prochain cours : Héritage et polymorphisme en Python.

Note pédagogique ENS

Transposition didactique : La notion d'objet peut être introduite dès le collège sans ordinateur par une activité débranchée : chaque élève reçoit une fiche "être vivant" avec des cases à remplir (nom, âge, couleur). La fiche vierge est la **classe**, la fiche remplie est l'**objet**. En NSI Première, on peut montrer que la classe `Eleve` du cours est similaire à un modèle de fiche élève qu'ils remplissent chaque année. Le parallèle avec le tableur (ligne = objet, colonne = attribut) facilite la compréhension.

Conseil pour l'enseignant formateur : Lors de la correction des TP, insistez sur la distinction classe/instance : beaucoup d'étudiants confondent `Eleve` (la classe) et `e` (l'instance). Un exercice efficace consiste à demander "Quel est le type de `e` ?" et "Quel est le type de `Eleve` ?". Proposez également un exercice où les étudiants doivent ajouter un attribut `date_naissance` et une méthode `age()` pour consolider la compréhension.

Évaluation formative : Proposez un défi codé en 10 minutes : "Créez une classe `Calculatrice` avec des méthodes `addition`, `soustraction`, `multiplication`, `division` (gérer la division par zéro)". Cela teste la compréhension des méthodes, des paramètres et du retour de valeur.

Prolongement : Les étudiants peuvent créer une classe `SalleDeClasse` qui contient une liste d'objets `Eleve` et des méthodes pour calculer la moyenne de la classe, lister les élèves par groupe, ou exporter les données au format CSV. Ce mini-projet prépare le projet intégratif de fin de module.

Templates : C:/Users/madan/Desktop/Supports de Cours
250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours

250626/Templatte/Presentation_BELACEL_V3.pptx

PDF: Cours_Informatique_ENS_2021_me_année_FR_03.pdf

Cours Informatique à l'ENS 2^{ème} année à l'École Supérieure 03 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'ENS 2^{ème} année à l'École Supérieure de France

Durée : 1h30

Année universitaire : 2025-2026

Fiche cours à l'Plateforme e-learning (Moodle)

Résumé : benchmark 10 universités algériennes + pratiques USTO/Biskra/M'sila.

Public visé : Étudiants ENS 2^{ème} année à l'École Supérieure de France

Compétences pré-requis : POO Python à l'classes, objets, méthodes (séance 02)

Objectifs transversaux :

- Appliquer la programmation orientée objet (classes, héritage, encapsulation) en Python
- Concevoir des algorithmes avancés (tri, recherche, récursivité) et estimer leur complexité
- Modéliser des données avec le modèle relationnel et requêtes SQL de base
- Comprendre l'architecture Web (HTML, CSS, JavaScript intro, HTTP)
- Utiliser Git et méthodes de travail collaboratif (branches, merge)
- Comparer l'enseignement de l'informatique au collège/lycée (Scratch, Python pédagogique)

Mode d'évaluation : TP POO + SQL 35 % à l'Projet intégré 40 % à l'Fiche séance pédagogique 25 %

Bibliographie : Guttag à l'*Introduction to Computation and Programming Using Python* ; Lutz à l'*Learning Python*

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Faire hériter une classe d'une autre classe
- Surcharger une méthode
- Utiliser `super()` pour appeler la classe parente
- Protéger des attributs avec l'encapsulation
- Expliquer le polymorphisme avec un exemple concret

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- Arrêté ministériel portant création des ENS (cycle Licence/Master) à l'définit le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.

- **Programme national d'Informatique des ENS algériennes** : fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** : détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** : cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** : étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Héritage : principe
2. Classe parente et classe fille
3. Surcharge de méthodes
4. La fonction `super()`
5. Héritage multiple (notion)
6. Encapsulation et propriétés
7. Polymorphisme
8. Exemple complet : hiérarchie scolaire

1. Héritage (15 min)

Principe : une classe peut hériter des attributs et méthodes d'une autre classe.

Vocabulaire :

- **Classe parente** (super-classe) : classe de base
- **Classe fille** (sous-classe) : hérite et peut ajouter/modifier

```
class Personne:
    def __init__(self, nom, prenom):
        self.nom = nom
        self.prenom = prenom

    def sePresenter(self):
        return f"{self.prenom} {self.nom}"

class Eleve(Personne): # Eleve hérite de Personne
    def __init__(self, nom, prenom, groupe):
        super().__init__(nom, prenom) # Appel du constructeur parent
        self.groupe = groupe

e = Eleve("Benali", "Sara", "ENS2-A")
print(e.sePresenter()) # Hérité ! → "Sara Benali"
print(e.groupe) # Propre à Eleve → "ENS2-A"

classDiagram
class Personne {
+string nom
+string prenom
+__init__(nom, prenom)
+sePresenter()
}
```

```

+description()
}
class Eleve {
+string groupe
+list notes
+__init__(nom, prenom, groupe)
+ajouter_note(note, matiere)
+moyenne_generale()
}
class Enseignant {
+string matiere
+__init__(nom, prenom, email, matiere)
+se_presenter()
}
class IntervenanteExterne {
+se_presenter()
}
Personne <|-- Eleve : h  rite
Personne <|-- Enseignant : h  rite
Personne <|-- IntervenanteExterne : h  rite
note for Personne "Super-classe"
note for Eleve "Sous-classe"
note for Enseignant "Surcharge se_presenter()"

```

2. Surcharge (override) (10 min)

Une classe fille peut red  finir une m  thode de la classe parente.

```

class Personne:
def description(self):
return "Une personne"

class Enseignant(Personne):
def __init__(self, nom, matiere):
self.nom = nom
self.matiere = matiere

def description(self): # Surcharge
return f"Enseignant de {self.matiere}"

p = Personne()
e = Enseignant("M. Belacel", "Informatique")
print(p.description()) # "Une personne"
print(e.description()) # "Enseignant de Informatique"

```

3. La fonction `super()` (10 min)

`super()` permet d'appeler une m  thode de la classe parente.

```

class Enseignant(Personne):
def __init__(self, nom, prenom, matiere):
super().__init__(nom, prenom) # Appelle Personne.__init__
self.matiere = matiere

def se_presenter(self):
#   tend la m  thode parente
base = super().se_presenter()
return f"{base}    Enseignant de {self.matiere}"

```

4. Héritage multiple (10 min)

Python permet d'hériter de plusieurs classes (attention aux conflits).

```
class Sportif:
    def discipline(self):
        return "Sport"

class Musicien:
    def instrument(self):
        return "Piano"

class EleveSportifMusicien(Personne, Sportif, Musicien):
    pass

e = EleveSportifMusicien("Ali", "Ben")
print(e.sePresenter()) # De Personne
print(e.discipline()) # De Sportif
print(e.instrument()) # De Musicien
```

5. Encapsulation (15 min)

Encapsulation : protéger l'accès aux attributs pour garantir l'intégrité.

```
class CompteBancaire:
    def __init__(self, titulaire, solde=0):
        self.titulaire = titulaire
        self._solde = 0 # Protéger (convention : _)
        if solde >= 0:
            self._solde = solde

    def déposer(self, montant):
        if montant > 0:
            self._solde += montant

    def retirer(self, montant):
        if 0 < montant <= self._solde:
            self._solde -= montant
        return montant
        return 0

    @property # Getter
    def solde(self):
        return self._solde

    @solde.setter # Setter
    def solde(self, valeur):
        if valeur >= 0:
            self._solde = valeur

# Utilisation
cb = CompteBancaire("Ali", 1000)
cb.deposer(500)
print(cb.solde) # 1500 (via le getter)
# cb._solde = 999999 # Techniquement possible mais déconseillé
```

Convention Python : `_attribut` = protégé, `__attribut` = privé (name mangling).

```
flowchart LR
    subgraph Encapsulation
        A["Attribut privé _solde"] --> B["Getteur @property"]
        A --> C["Setteur @solde.setter"]
        B --> D["Lecture contrôlée"]
        C --> E["Écriture avec validation"]
    end
```

```

end
subgraph Polymorphisme
F[Fonction faire_cours] --> G[Enseignant.se_presenter]
F --> H[IntervenantExterne.se_presenter]
G --> I[Même interface<br/>comportements différents]
H --> I
end

```

6. Polymorphisme (10 min)

Polymorphisme : la même interface peut avoir des comportements différents selon le type d'objet.

```

def faire_cours(intervenant):
    """Fonction polymorphe"""
    print(intervenant.se_presenter())
    print(" va faire cours.")

class Enseignant(Personne):
    def se_presenter(self):
        return f"{self.prenom} {self.nom} (enseignant)"

class IntervenantExterne(Personne):
    def se_presenter(self):
        return f"{self.prenom} {self.nom} (intervenant externe)"

# Même fonction, comportements différents
faire_cours(Enseignant("Belacel", "Madani"))
faire_cours(IntervenantExterne("Dupont", "Jean"))

```

7. Exemple complet : système scolaire (15 min)

```

class Personne:
    def __init__(self, nom, prenom, email):
        self.nom = nom
        self.prenom = prenom
        self.email = email

    def __str__(self):
        return f"{self.prenom} {self.nom}"

class Eleve(Personne):
    def __init__(self, nom, prenom, email, groupe):
        super().__init__(nom, prenom, email)
        self.groupe = groupe
        self.notes = []

    def ajouter_note(self, note, matiere):
        self.notes.append({"matiere": matiere, "note": note})

    def moyenne_generale(self):
        if not self.notes:
            return 0.0
        return sum(n["note"] for n in self.notes) / len(self.notes)

class Enseignant(Personne):
    def __init__(self, nom, prenom, email, matiere):
        super().__init__(nom, prenom, email)
        self.matiere = matiere

    def __str__(self):
        return f"{self.prenom} {self.nom} â€” {self.matiere}"

```

```
# Test
eleve = Eleve("Benali", "Sara", "s.benali@etu.ens.dz", "ENS2-A")
eleve.ajouter_note(15, "Info")
eleve.ajouter_note(12, "Fran ais")
prof = Enseignant("Belacel", "Madani", "m.belacel@ens.dz", "Informatique")
print(eleve) # "Sara Benali"
print(prof) # "Madani Belacel @  Informatique"
print(f"Moyenne : {eleve.moyenne_generale():.2f}")
```

QCM d'auto- valuation

1. Que permet l'h ritage en POO ?

a) Cr er une classe   partir d'une autre classe    b) Effacer des attributs c) Emp  cher la modification du code

2. Quel mot-cl  permet d'appeler le constructeur de la classe parente ?

a) `parent()` b) `super()`    c) `base()`

3. L'encapsulation permet de :

a) Rendre le code plus rapide b) Prot ger l'acc s aux attributs    c) Supprimer les m thodes

4. Que signifie le polymorphisme ?

a) Une classe peut avoir plusieurs parents b) La m  me interface peut avoir diff rents comportements selon l'objet    c) Le code s'ex cute plus lentement

5. Quelle convention Python pour un attribut prot g  ?

a) `_attribut`    b) `__attribut__` c) `attribut_`

Conclusion

H ritage, encapsulation et polymorphisme sont les trois piliers de la POO. Ils permettent d' crire du code structur , r utilisable et extensible.

Prochain cours : Algorithmes avanc s et r cursivit .

Note p dagogique ENS

Transposition didactique : L'h ritage peut  tre expliqu  au coll ge avec un exemple simple : "Un carr  est un rectangle particulier"    le carr  h rite des propri t s du rectangle (4 c t s, angles droits) mais ajoute la contrainte d' galit  des c t s. En NSI Terminale, la hi rarchie `Personne`    `Eleve/Enseignant` illustre parfaitement le concept. L'encapsulation se traduit p dagogiquement par la m taphore de la bo te noire : on utilise un objet sans conna tre son fonctionnement interne.

Conseil pour l'enseignant formateur : L'h ritage multiple est souvent source de confusion. Insistez d'abord sur l'h ritage simple avec des exemples concrets du monde scolaire. Pour l'encapsulation, l'exemple du `CompteBancaire` est efficace car les  tudiants comprennent intuitivement qu'on ne doit pas pouvoir modifier un solde directement. Proposez un mini-TP o  les  tudiants doivent ajouter une classe `ProfesseurPrincipal(Enseignant)` qui h rite d'`Enseignant` et ajoute un attribut `classe`.

Prolongement : Les Étudiants peuvent modéliser un système complet de gestion d'établissement scolaire avec une hiérarchie de classes : `Personne` à `Eleve`, `Enseignant`, `PersonnelAdministratif`. Chaque sous-classe ajoute des attributs et méthodes spécifiques. Ce modèle POO sera utilisé dans le projet ENS2 et la base de données SQL.

Templates : C:/Users/madan/Desktop/Supports de Cours
250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours
250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_2ème_année_FR_04.pdf

Cours Informatique à l'ENS 2^{ème} année S^éance 04 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'ENS 2^{ème} année D^épartement de français

Durée : 1h30

Année universitaire : 2025-2026

Fiche cours à l'Plateforme e-learning (Moodle)

Résumé : benchmark 10 universités algériennes + pratiques USTO/Biskra/M'sila.

Public visé : Étudiants ENS 2^{ème} année D^épartement Français

Compétences pré-requis : Algorithmique ENS1 (séance 05), complexité $O(n)$, $O(n^2)$

Objectifs transversaux :

- Appliquer la programmation orientée objet (classes, héritage, encapsulation) en Python
- Concevoir des algorithmes avancés (tri, recherche, récursivité) et estimer leur complexité
- Modéliser des données avec le modèle relationnel et requêtes SQL de base
- Comprendre l'architecture Web (HTML, CSS, JavaScript intro, HTTP)
- Utiliser Git et méthodes de travail collaboratif (branches, merge)
- Comparer l'enseignement de l'informatique au collège/lycée (Scratch, Python pédagogique)

Mode d'évaluation : TP POO + SQL 35 % à l'Projet intégrée 40 % à l'Fiche séance pédagogique 25 %

Bibliographie : Cormen et al. à l'*Introduction to Algorithms* ; Downey à l'*Think Python* (chapitre récursivité)

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Expliquer la récursivité avec un exemple
- Écrire une fonction récursive simple (factorielle, Fibonacci)
- Implémenter le tri bulle et le tri fusion (notion)
- Implémenter la recherche dichotomique
- Comparer la complexité des algorithmes de tri

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** à l'définit le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.

- **Programme national d'Informatique des ENS algériennes** : fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** : détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** : cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** : étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Récursivité : principe et exemples
2. Récursivité vs itération
3. Recherche dichotomique
4. Tri bulle
5. Tri fusion (merge sort)
6. Comparaison des tris

1. Récursivité (15 min)

Fonction récursive : fonction qui s'appelle elle-même.

Deux éléments essentiels :

- **Cas de base** : condition d'arrêt (n'utilise pas la récursion)
- **Appel récursif** : la fonction se rappelle avec un paramètre modifié

```
flowchart TD
    subgraph Récursivité
        A[factorielle 5] --> B[5 * factorielle 4]
        B --> C[4 * factorielle 3]
        C --> D[3 * factorielle 2]
        D --> E[2 * factorielle 1]
        E --> F[1 cas de base]
        F --> G[Retour 1]
        G --> H[Retour 2]
        H --> I[Retour 6]
        I --> J[Retour 24]
        J --> K[Résultat 120]
    end
```

Exemple : factorielle

```
def factorielle(n):
    if n <= 1: # Cas de base
        return 1
    return n * factorielle(n - 1) # Appel récursif

print(factorielle(5)) # 120 = 5*4*3*2*1
```

Exemple : Fibonacci

```
def fibonacci(n):
    if n <= 1: # Cas de base
        return n
    return fibonacci(n-1) + fibonacci(n-2) # Double appel

print(fibonacci(10)) # 55
```

Exercice rapide : Suite de Syracuse (5 min)

La conjecture de Syracuse (suite de Collatz) est définie par :

- $u_0 = n$ (entier positif)
- Si u_k est pair : $u_{k+1} = u_k / 2$
- Si u_k est impair : $u_{k+1} = 3 * u_k + 1$

Implémentez une fonction récursive `syracuse(n)` qui affiche les termes de la suite jusqu'à 1.

```
def syracuse(n):
    print(n, end=" ")
    if n == 1: # Cas de base
        return
    if n % 2 == 0: # Pair
        syracuse(n // 2)
    else: # Impair
        syracuse(3 * n + 1)

syracuse(6) # 6 3 10 5 16 8 4 2 1
```

Question : Tous les nombres finissent-ils par atteindre 1 ? C'est une conjecture non encore démontrée.

2. Récursivité vs itération (10 min)

```
# Itératif (boucle)
def factorielle_iter(n):
    resultat = 1
    for i in range(2, n + 1):
        resultat *= i
    return resultat

# Récursif
def factorielle_rec(n):
    if n <= 1:
        return 1
    return n * factorielle_rec(n - 1)
```

Critère	Itératif	Récursif
Lisibilité	Simple	Très lisible (mathématique)
Performance	Plus rapide	Plus lent (appels empilés)
Mémoire	Constante	$O(n)$ pile d'appels
Usage	Boucles simples	Problèmes divisibles

Activité débranchée : Récursivité avec des miroirs (5 min)

Prenez deux petits miroirs face à face : l'image se répète à l'infini à l'échelle d'une réflexion visuelle ! Chaque reflet est un appel récursif, l'embodiment est la pile d'appels.

En classe : Les poupées russes (matriochkas) illustrent parfaitement la récursivité : chaque poupée en contient une plus petite.

3. Recherche dichotomique (15 min)

Principe : dans un tableau trié, comparer avec l'élément du milieu, puis chercher à gauche ou à droite.

```
def recherche_dichotomique(tableau, cible):
    debut = 0
    fin = len(tableau) - 1

    while debut <= fin:
        milieu = (debut + fin) // 2
        if tableau[milieu] == cible:
            return milieu # Trouvé
        elif tableau[milieu] < cible:
            debut = milieu + 1 # Chercher à droite
        else:
            fin = milieu - 1 # Chercher à gauche

    return -1 # Non trouvé

# Test
notes = [5, 8, 10, 12, 15, 18, 20]
print(recherche_dichotomique(notes, 15)) # 4
print(recherche_dichotomique(notes, 7)) # -1
```

Complexité : $O(\log n)$ est beaucoup plus rapide que la recherche linéaire $O(n)$.

```
flowchart TD
    subgraph RechercheDichotomique
        A["Tableau trié<br/>notes = [5, 8, 10, 12, 15, 18, 20]"] --> B["Cible = 15"]
        B --> C["debut=0, fin=6<br/>milieu=3<br/>tableau[3]=12"]
        C --> D["12 < 15 | debut = milieu+1 = 4"]
        D --> E["debut=4, fin=6<br/>milieu=5<br/>tableau[5]=18"]
        E --> F["18 > 15 | fin = milieu-1 = 4"]
        F --> G["debut=4, fin=4<br/>milieu=4<br/>tableau[4]=15"]
        G --> H["15 == 15 | Trouvé à l'indice 4"]
        H --> I["fin = milieu-1"]
        I --> J["12 < cible | fin = milieu-1"]
        J --> K["18 < cible | debut = milieu+1"]
        K --> L["end"]
    end
```

Mini-exercice : Recherche dichotomique récursive (5 min)

Écrivez une version récursive de la recherche dichotomique.

```
def recherche_dichotomique_recursive(tableau, cible, debut=0, fin=None):
    if fin is None:
        fin = len(tableau) - 1
    if debut > fin:
        return -1
    milieu = (debut + fin) // 2
    if tableau[milieu] == cible:
        return milieu
    elif tableau[milieu] < cible:
        return recherche_dichotomique_recursive(tableau, cible, milieu + 1, fin)
    else:
        return recherche_dichotomique_recursive(tableau, cible, debut, milieu - 1)

notes = [3, 8, 10, 12, 14, 15, 18, 20]
print(recherche_dichotomique_recursive(notes, 14)) # 4
```

Lien avec l'enseignement : Le jeu "plus grand, plus petit" est une recherche dichotomique naturelle.

4. Tri bulle (15 min)

Principe : parcourir le tableau, comparer les éléments adjacents, les permuter s'ils sont dans le mauvais ordre.

```
def tri_bulle(tableau):
    n = len(tableau)
    for i in range(n):
        trie = True
        for j in range(n - i - 1):
            if tableau[j] > tableau[j + 1]:
                tableau[j], tableau[j + 1] = tableau[j + 1], tableau[j]
            trie = False
        if trie: # Optimisation : d'ici trié
            break
    return tableau

# Test
notes = [15, 8, 12, 20, 5]
print(tri_bulle(notes)) # [5, 8, 12, 15, 20]
```

Complexité : $O(n^2)$ dans le pire cas, $O(n)$ si d'ici trié.

Mini-exercice : Trier des mots par ordre alphabétique (5 min)

```
def tri_bulle_mots(mots):
    n = len(mots)
    for i in range(n):
        for j in range(n - i - 1):
            if mots[j] > mots[j + 1]:
                mots[j], mots[j + 1] = mots[j + 1], mots[j]
    return mots

prenoms = ["Zinedine", "Ali", "Sara", "Mohamed", "Aicha"]
print(tri_bulle_mots(prenoms))
# ['Aicha', 'Ali', 'Mohamed', 'Sara', 'Zinedine']
```

Remarque : Python compare les chaînes par ordre lexicographique (table ASCII/Unicode).

5. Tri fusion (merge sort) (15 min)

Principe : diviser le tableau en deux, trier chaque moitié récursivement, puis fusionner.

```
def tri_fusion(tableau):
    if len(tableau) <= 1:
        return tableau

    milieu = len(tableau) // 2
    gauche = tri_fusion(tableau[:milieu])
    droite = tri_fusion(tableau[milieu:])

    return fusionner(gauche, droite)

def fusionner(gauche, droite):
    resultat = []
    i = j = 0
    while i < len(gauche) and j < len(droite):
        if gauche[i] <= droite[j]:
            resultat.append(gauche[i])
            i += 1
        else:
            resultat.append(droite[j])
            j += 1
    resultat.extend(gauche[i:])
    resultat.extend(droite[j:])
    return resultat
```

```

else:
    resultat.append(droite[j])
    j += 1
    resultat.extend(gauche[i:])
    resultat.extend(droite[j:])
    return resultat

# Test
notes = [15, 8, 12, 20, 5]
print(tri_fusion(notes)) # [5, 8, 12, 15, 20]

```

Complexité : $O(n \log n)$ dans tous les cas "plus efficace que le tri bulle".

Visualisation du tri fusion

```

graph TD
    subgraph TriFusion
    A[15, 8, 12, 20, 5] --> B[15, 8, 12]
    A --> C[20, 5]
    B --> D[15, 8]
    B --> E[12]
    D --> F[15]
    D --> G[8]
    C --> H[20]
    C --> I[5]
    F --> J[8, 15]
    G --> J
    J --> K[8, 12, 15]
    E --> K
    H --> L[5, 20]
    I --> L
    K --> M[5, 8, 12, 15, 20]
    L --> M
    end

```

Principe "diviser pour régner" : Le problème est divisé en sous-problèmes (diviser) jusqu'à obtenir des cas triviaux, puis les solutions sont combinées (fusionner).

6. Comparaison des tris (10 min)

Algorithme	Pire cas	Meilleur cas	Mémoire	Stable
Tri bulle	$O(n^2)$	$O(n)$	$O(1)$	Oui
Tri fusion	$O(n \log n)$	$O(n \log n)$	$O(n)$	Oui
Tri rapide	$O(n^2)$	$O(n \log n)$	$O(\log n)$	Non
Tri insertion	$O(n^2)$	$O(n)$	$O(1)$	Oui

Pour un enseignant :

- Tableau de 10 notes : tri bulle suffit
- Classe de 30 élèves : tri fusion
- Base de 10 000 enregistrements : tri fusion ou rapide

6bis. Algorithmes dans l'enseignement secondaire (10 min)

Programmes scolaires :

Niveau	Algorithmes abordés	Langage
Collège (4 ^{ème} /3 ^{ème})	Algorithmes simples (moyenne, min/max)	Scratch
SNT Seconde	Recherche, tris simples	Python
NSI Premièrre	Tri bulle, tri insertion, récursivité	Python
NSI Terminale	Tri fusion, tri rapide, dichotomique	Python

Conseils pour l'enseignant :

- **Collège** : privilégiez l'approche débranchée (cartes, manipulation physique)
- **Lycée** : alternez implémentation Python et analyse de complexité
- **Autoévaluation** : faites tracer l'exécution sur papier avant de coder

Exemple d'activité pour le collège :

1. Distribuez 10 cartes numérotées dans le désordre
2. Les élèves les trient avec la méthode du tri bulle
3. Comptez les échanges, recommencez avec 20 cartes

Repères historiques

Année	Algorithme	Inventeur
~200 av. J.-C.	Crible d'Ératosthène	Ératosthène
1945	Tri fusion	John von Neumann
1959	Tri rapide	Tony Hoare
1960	Recherche dichotomique (formalisée)	D. H. Lehmer
1970	Tri bulle (popularisée)	Donald Knuth

Le saviez-vous ? Les algorithmes de tri sont exécutés des milliards de fois par jour (bases de données, moteurs de recherche, e-commerce).

QCM d'autoévaluation

1. Quel est le cas de base de la fonction factorielle ?
a) $n == 0$ b) $n \leq 1$ c) $n == 2$
2. Quelle est la complexité de la recherche dichotomique ?
a) $O(n)$ b) $O(\log n)$ c) $O(n^2)$
3. Le tri bulle a une complexité de :
a) $O(n \log n)$ b) $O(n^2)$ dans le pire cas c) $O(1)$
4. Quel algorithme de tri est récursif et a une complexité $O(n \log n)$?
a) Tri bulle b) Tri fusion c) Tri insertion
5. Quelle structure de données est nécessaire pour la recherche dichotomique ?

a) Une liste chaînée b) Un tableau trié c) Une pile

6. Quelle est la complexité de Fibonacci récursive naïve ?

a) $O(n)$ b) $O(2^n)$ c) $O(n^2)$

7. Quel algorithme utilise la stratégie "diviser pour régner" ?

a) Tri bulle b) Tri fusion c) Tri insertion

8. À quoi sert la variable `trie` dans le tri bulle optimisé ?

a) Compter les itérations b) Détecter si déjà trié c) Permuter les éléments

7. Conclusion (5 min)

La récursivité permet d'écire des algorithmes élégants pour des problèmes divisibles. Les algorithmes de tri et de recherche sont fondamentaux et leur complexité détermine leur efficacité.

Prochain cours : Bases de données et SQL.

Note pédagogique ENS

Transposition didactique : La récursivité peut être introduite au collège avec les poupées russes ou les miroirs. En NSI Première, factorielle et Fibonacci sont des classiques. Le tri fusion illustre "diviser pour régner", transposable à d'autres disciplines. La recherche dichotomique est le jeu du "plus grand/plus petit".

Conseil pour l'enseignant formateur : Les étudiants confondent souvent récursivité et boucle. Insistez sur la différence : la récursivité divise le problème, l'itération répète une action. Faites tracer `factorielle(4)` sur papier.

Section de révision : Proposez un tableau vierge à remplir (nom, complexité, usage pédagogique). Complétez-le collectivement.

Auto-évaluation (exit ticket) : Chaque étudiant écrit : (1) une chose comprise, (2) une chose floue, (3) une question pour le prochain cours.

Prolongement : Implémentez le tri rapide et comparez ses performances au tri fusion avec `matplotlib`.

Templates : C:/Users/madan/Desktop/Supports de Cours 250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_2ème_année_FR_05.pdf

Cours Informatique à l'ENS 2^{ème} année à l'École Supérieure 05 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'ENS 2^{ème} année à l'École Supérieure de France

Durée : 1h30

Année universitaire : 2025-2026

Fiche cours à l'Plateforme e-learning (Moodle)

Résumé : benchmark 10 universités algériennes + pratiques USTO/Biskra/M'sila.

Public visé : Étudiants ENS 2^{ème} année à l'École Supérieure de France

Compétences pré-requises : Algorithmique (séance 04), POO Python (séances 02-03)

Objectifs transversaux :

- Appliquer la programmation orientée objet (classes, héritage, encapsulation) en Python
- Concevoir des algorithmes avancés (tri, recherche, récursivité) et estimer leur complexité
- Modéliser des données avec le modèle relationnel et requêtes SQL de base
- Comprendre l'architecture Web (HTML, CSS, JavaScript intro, HTTP)
- Utiliser Git et méthodes de travail collaboratif (branches, merge)
- Comparer l'enseignement de l'informatique au collège/lycée (Scratch, Python pédagogique)

Mode d'évaluation : TP POO + SQL 35 % à l'Projet intégré 40 % à l'Fiche séance pédagogique 25 %

Bibliographie : Connolly & Begg à l'*Database Systems* ; Beaulieu à l'*Learning SQL* (O'Reilly)

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Expliquer le modèle relationnel (tables, lignes, colonnes)
- Créer une table avec SQLite
- Écrire des requêtes SELECT, WHERE, JOIN
- Insérer et mettre à jour des données
- Comprendre la notion de clé primaire et étrangère

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** à l'définit le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algériennes** à l'fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.

- **Arrêté 712 fixant le programme des concours ENS** : définit les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algébrien** : cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** : étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Pourquoi une base de données ?
2. Modèle relationnel
3. SQLite et Python
4. CREATE TABLE
5. INSERT, SELECT, WHERE
6. Mise à jour et suppression
7. Clés étrangères et JOIN
8. Exemple : gestion de notes

1. Pourquoi une base de donn  es ? (10 min)

Limites d'un fichier CSV :

- Pas de structure stricte
- Pas de validation (notes > 20 possibles)
- Difficile de faire des liens entre tables
- Pas de concurrence (deux personnes ne peuvent pas écrire en même temps)

Base de données relationnelle :

- Structure rigide (schéma)
- Contraintes (clés, types, valeurs)
- Requêtes puissantes (langage SQL)
- Intégrité des données

2. Modèle relationnel (10 min)

Table : ensemble de lignes (enregistrements) et colonnes (champs).

Table Eleven

[illegible]

â", id â", nom â", prenom â", groupe â",

a"æâ"eā"eà"eä"eå"¼ã"eâ"eă"eâ"eă"eâ"eă"eâ"eă"¼ã"eâ"eă"eâ"eă"eâ"eă"eâ"eă"¼ã"eâ"eă"eâ"eă"
eâ"eă"eâ"eă"eâ"eă"eâ"eă"¼

© 2007 Pearson Education, Inc.

â", 1 â", Belacel â", Ali â", ENS2-A â".

â", 2 â", Benali â", Sara â", ENS2-A â".

â", 3 â", Cherif â", Mohamed â", ENS2-B â",

[illegible]

eâ"eâ"eâ"eâ"eâ"eâ"eâ"~

Vocabulaire :

- **ClÃ© primaire** : colonne (ou combinaison) qui identifie chaque ligne de faÃ§on unique
- **ClÃ© Ã©trangÃ¨re** : colonne qui rÃ©fÃ©rence la clÃ© primaire d'une autre table

```
erDiagram
ELEVES {
    int id PK
    string nom
    string prenom
    string groupe
}
MATIERES {
    int id PK
    string nom
    float coefficient
}
NOTES {
    int id PK
    int eleve_id FK
    int matiere_id FK
    float valeur
}
ELEVES ||--o{ NOTES : possede
MATIERES ||--o{ NOTES : concerne
```

3. SQLite et Python (10 min)

SQLite est une base de donnÃ©es lÃ©gÃ¨re, intÃ©grÃ©e Ã Python (pas d'installation serveur).

```
import sqlite3

# Connexion (crÃ©e le fichier .db)
conn = sqlite3.connect("ecole.db")
curseur = conn.cursor()

# ExÃ©cuter une requÃªte
curseur.execute("""
CREATE TABLE IF NOT EXISTS eleves (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom TEXT NOT NULL,
    prenom TEXT NOT NULL,
    groupe TEXT
)
""")

conn.commit() # Valider
conn.close() # Fermer
```

4. CREATE TABLE (10 min)

```
conn = sqlite3.connect("ecole.db")
c = conn.cursor()

c.execute("""
CREATE TABLE IF NOT EXISTS matieres (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom TEXT NOT NULL,
    coefficient REAL DEFAULT 1.0

```

```

)
"""

c.execute("""
CREATE TABLE IF NOT EXISTS notes (
id INTEGER PRIMARY KEY AUTOINCREMENT,
eleve_id INTEGER NOT NULL,
matiere_id INTEGER NOT NULL,
valeur REAL NOT NULL CHECK(valeur >= 0 AND valeur <= 20),
FOREIGN KEY (eleve_id) REFERENCES eleves(id),
FOREIGN KEY (matiere_id) REFERENCES matieres(id)
)
""")

conn.commit()

```

5. INSERT, SELECT, WHERE (15 min)

INSERT

```

c.execute("INSERT INTO eleves (nom, prenom, groupe) VALUES (?, ?, ?)",
("Belacel", "Ali", "ENS2-A"))
c.execute("INSERT INTO matieres (nom, coefficient) VALUES (?, ?)",
("Informatique", 3.0))

```

SELECT

```

# Tous les élèves
c.execute("SELECT * FROM eleves")
for ligne in c.fetchall():
    print(ligne)

# Avec condition
c.execute("SELECT * FROM eleves WHERE groupe = ?", ("ENS2-A",))

flowchart LR
A[Client Python] --> B[Connexion SQLite]
B --> C[Curseur]
C --> D[execute SQL]
D --> E{Type requete ?}
E -->|SELECT| F[fetchall / fetchone]
E -->|INSERT/UPDATE/DELETE| G[commit]
F --> H[Traitement resultats]
G --> H
H --> I[close]

```

6. Mise à jour et suppression (10 min)

```

# UPDATE
c.execute("UPDATE eleves SET groupe = ? WHERE nom = ?",
("ENS2-B", "Belacel"))

# DELETE
c.execute("DELETE FROM notes WHERE valeur < ?", (0,)) # Aucun effet

# Attention : DELETE sans WHERE supprime TOUT

```

7. Clauses étrangères et JOIN (15 min)

```
# Insérer une note
c.execute("""
INSERT INTO notes (eleve_id, matiere_id, valeur)
VALUES (?, ?, ?)
""", (1, 1, 15))

# Jointure : notes + noms des élèves + noms des matières
c.execute("""
SELECT e.nom, e.prenom, m.nom, n.valeur
FROM notes n
JOIN eleves e ON n.eleve_id = e.id
JOIN matieres m ON n.matiere_id = m.id
""")

for ligne in c.fetchall():
    print(f"{ligne[0]} {ligne[1]} â€” {ligne[2]} : {ligne[3]}")
```

Résultat :

```
Belacel Ali â€” Informatique : 15.0
```

8. Exemple complet : relevé de notes (15 min)

```
import sqlite3

conn = sqlite3.connect("ecole.db")
c = conn.cursor()

# Création des tables
c.execute("""
CREATE TABLE IF NOT EXISTS eleves (
id INTEGER PRIMARY KEY AUTOINCREMENT,
nom TEXT, prenom TEXT, groupe TEXT
)
""")
c.execute("""
CREATE TABLE IF NOT EXISTS matieres (
id INTEGER PRIMARY KEY AUTOINCREMENT,
nom TEXT, coefficient REAL
)
""")
c.execute("""
CREATE TABLE IF NOT EXISTS notes (
id INTEGER PRIMARY KEY AUTOINCREMENT,
eleve_id INTEGER, matiere_id INTEGER, valeur REAL,
FOREIGN KEY(eleve_id) REFERENCES eleves(id),
FOREIGN KEY(matiere_id) REFERENCES matieres(id)
)
""")

# Insertion des données
c.execute("INSERT INTO eleves VALUES (1, 'Ali', 'Belacel', 'ENS2-A')")
c.execute("INSERT INTO matieres VALUES (1, 'Informatique', 3.0)")
c.execute("INSERT INTO matieres VALUES (2, 'Français', 2.0)")
c.execute("INSERT INTO notes VALUES (NULL, 1, 1, 15)")
c.execute("INSERT INTO notes VALUES (NULL, 1, 2, 12)")

# Relevé de notes avec moyenne
c.execute("""
SELECT e.nom, e.prenom, m.nom, n.valeur, m.coefficient
FROM notes n
JOIN eleves e ON n.eleve_id = e.id
JOIN matieres m ON n.matiere_id = m.id
```

```

WHERE e.id = 1
""")

print("=== Relevé de notes ===")
total_pondere = 0
total_coef = 0
for ligne in c.fetchall():
    print(f"{ligne[0]} {ligne[1]} : {ligne[2]} (coef {ligne[4]})")
    total_pondere += ligne[3] * ligne[4]
    total_coef += ligne[4]

moyenne = total_pondere / total_coef if total_coef > 0 else 0
print(f"Moyenne globale : {moyenne:.2f}")

conn.commit()
conn.close()

```

QCM d'auto-évaluation

1. Quel langage permet d'interroger une base de données relationnelle ?

a) HTML b) SQL c) Python

2. Une clé primaire doit être :

a) Unique et non nulle b) Optionnelle c) Toujours de type texte

3. Quelle clause SQL filtre les résultats ?

a) SELECT b) WHERE c) JOIN

4. Que fait `DELETE FROM eleves` ?

a) Supprime la table b) Supprime toutes les lignes c) Supprime une colonne

5. Une jointure (JOIN) permet de :

a) Combiner deux tables b) Trier les résultats c) Créer une nouvelle base

Conclusion

Les bases de données relationnelles et SQL sont au cœur de la gestion des données structurées. SQLite, intégré à Python, permet de prototyper rapidement une solution complète (création, insertion, interrogation, jointure) sans installation serveur. Les concepts de clé primaire, clé étrangère et jointure sont universels et se retrouvent dans tous les SGBD (MySQL, PostgreSQL, Oracle).

Prochain cours : Développement Web avec HTML, CSS et JavaScript.

Note pédagogique ENS

Transposition didactique : Les bases de données peuvent être enseignées au collège et au lycée sans ordinateur, par des activités débranchées :

- **Fichier papier :** chaque élève reçoit une fiche (table) avec des champs à remplir (nom, prénom, date de naissance). On simule une requête SELECT en triant les fiches.

- **Jeu de cartes** : des cartes "Ã©lÃ©ve" et des cartes "note" sont reliÃ©es par un trombone (clÃ© Ã©trangÃ©re). Les Ã©lÃ©ves doivent apparier les cartes (JOIN).
- **En SNT Seconde** (thÃ©me "DonnÃ©es structurÃ©es") : faire rÃ©flÃ©chir les Ã©lÃ©ves sur la diffÃ©rence entre un fichier texte non structurÃ© et un tableau structurÃ© (feuille de calcul). Le parallÃ©le avec un tableur (Excel, LibreOffice Calc) est immÃ©diat.
- **En NSI PremiÃ©re** : SQL fait partie intÃ©grante du programme. L'exemple du relevÃ© de notes (section 8) peut Ãªtre directement repris en classe.

Conseil pour l'enseignant formateur : Demander aux Ã©tudiants ENS de prÃ©parer une fiche pÃ©dagogique de 45 minutes sur les bases de donnÃ©es pour une classe de 4e ou de Seconde, en intÃ©grant une activitÃ© dÃ©branchÃ©e et une activitÃ© numÃ©rique. Cela favorise la transposition didactique, compÃ©tence clÃ© du mÃ©tier d'enseignant.

Prolongement : Les Ã©tudiants peuvent crÃ©er une mini-application de gestion de bibliothÃ©que (livres, auteurs, emprunts) avec SQLite et Python, et rÃ©flÃ©chir aux requÃªtes utiles pour un enseignant (gestion des notes, des absences, des groupes).

Templates : C:/Users/madan/Desktop/Supports de Cours 250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_2Ã©me_annÃ©e_FR_06.pdf

Cours Informatique à l'ENS 2^{ème} année à l'École Supérieure 06 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'ENS 2^{ème} année à l'École Supérieure de France

Durée : 1h30

Année universitaire : 2025-2026

Fiche cours à l'Plateforme e-learning (Moodle)

Résumé : benchmark 10 universités algériennes + pratiques USTO/Biskra/M'sila.

Public visé : Étudiants ENS 2^{ème} année à l'École Supérieure de France

Compétences pré-requis : Réseaux (ENS1 séance 07), fichiers Python (ENS1 séance 09)

Objectifs transversaux :

- Appliquer la programmation orientée objet (classes, héritage, encapsulation) en Python
- Concevoir des algorithmes avancés (tri, recherche, récursivité) et estimer leur complexité
- Modéliser des données avec le modèle relationnel et requêtes SQL de base
- Comprendre l'architecture Web (HTML, CSS, JavaScript intro, HTTP)
- Utiliser Git et méthodes de travail collaboratif (branches, merge)
- Comparer l'enseignement de l'informatique au collège/lycée (Scratch, Python pédagogique)

Mode d'évaluation : TP POO + SQL 35 % à l'Projet intégré 40 % à l'Fiche séance pédagogique 25 %

Bibliographie : MDN Web Docs (developer.mozilla.org) ; Duckett à l'HTML & CSS

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Créer une page HTML structurée (head, body, sections)
- Appliquer des styles CSS (couleurs, polices, mise en page)
- Ajouter un script JavaScript simple (interaction utilisateur)
- Expliquer le cycle requête/réponse HTTP
- Comprendre le rôle des formulaires Web

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** à l'définit le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algériennes** à l'fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.

- **Arrêté 712 fixant le programme des concours ENS** : détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** : cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** : étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Architecture Web (client-serveur)
2. HTML : structure d'une page
3. CSS : mise en forme
4. JavaScript : interaction
5. Formulaires Web
6. Déploiement et hébergement
7. Mini-projet : page personnelle enseignant

1. Architecture Web (10 min)

Client-serveur : le navigateur (client) envoie une requête HTTP au serveur, qui répond avec le contenu.

```
sequenceDiagram
    participant Client as Navigateur
    participant Serveur as Serveur Web
    participant BD as Base de données

    Client->>Serveur: GET /index.html
    Serveur-->>Client: 200 OK [HTML]
    Client->>Serveur: GET /style.css
    Serveur-->>Client: 200 OK [CSS]
    Client->>Serveur: POST /envoyer (formulaire)
    Serveur->>BD: INSERT INTO messages
    BD-->>Serveur: OK
    Serveur-->>Client: 301 Redirect /merci
    Client->>Serveur: GET /merci
    Serveur-->>Client: 200 OK [HTML]
```

URL : <https://www.ens-mostaganem.dz/departements/francais>

- [https](https://) : protocole sécurisé
- www.ens-mostaganem.dz : nom de domaine
- [/departements/francais](https://www.ens-mostaganem.dz/departements/francais) : chemin

2. HTML (20 min)

HTML (HyperText Markup Language) utilise des balises pour structurer le contenu.

```
<!DOCTYPE html>
<html lang="fr">
<head>
```

```

<meta charset="UTF-8">
<title>Mon Portfolio â€” Enseignant</title>
</head>
<body>
<header>
<h1>BELACEL Madani</h1>
<nav>
<a href="#cours">Cours</a>
<a href="#recherche">Recherche</a>
<a href="#contact">Contact</a>
</nav>
</header>

<main>
<section id="cours">
<h2>Mes cours</h2>
<ul>
<li>Informatique ENS 1re annÃ©e</li>
<li>Informatique ENS 2eme annÃ©e</li>
<li>TIC â€” Licence</li>
</ul>
</section>

<section id="recherche">
<h2>Recherche</h2>
<table>
<tr><th>AnnÃ©e</th><th>Publication</th></tr>
<tr><td>2025</td><td>TICE en AlgÃ©rie</td></tr>
</table>
</section>
</main>

<footer>
<p>Â© 2026 BELACEL Madani â€” ENS Mostaganem</p>
</footer>
</body>
</html>

```

Balises sémantiques : <header>, <nav>, <main>, <section>, <footer> (meilleur référencement et accessibilité).

```

graph TD
A[html] --> B[head]
A --> C[body]
B --> D[meta charset]
B --> E[title]
C --> F[header]
C --> G[main]
C --> H[footer]
F --> I[h1]
F --> J[nav]
J --> K[a href]
J --> L[a href]
G --> M[section id=cours]
G --> N[section id=recherche]
M --> O[h2]
M --> P[ul]
P --> Q[li]
P --> R[li]
N --> S[table]
S --> T[tr]
T --> U[th]
T --> V[th]
H --> W[p]

```

```
style A fill:#003366,color:#fff
style B fill:#003366,color:#fff
style C fill:#003366,color:#fff
```

3. CSS (15 min)

CSS (Cascading Style Sheets) contrôle l'apparence.

```
/* style.css */
body {
  font-family: 'Segoe UI', Arial, sans-serif;
  margin: 0;
  padding: 20px;
  background-color: #f5f5f5;
  color: #333;
}

header {
  background-color: #003366;
  color: white;
  padding: 20px;
  text-align: center;
}

nav a {
  color: white;
  margin: 0 10px;
  text-decoration: none;
}

nav a:hover {
  text-decoration: underline;
}

table {
  border-collapse: collapse;
  width: 100%;
}

th, td {
  border: 1px solid #ddd;
  padding: 8px;
  text-align: left;
}

th {
  background-color: #003366;
  color: white;
}
```

Intégration : `<link rel="stylesheet" href="style.css">` dans le `<head>`.

4. JavaScript (15 min)

JavaScript permet l'interactivité côté client.

```
<!DOCTYPE html>
<html>
<body>
<h2>Calculateur de moyenne</h2>
```

```

<input type="number" id="note1" placeholder="Note 1">
<input type="number" id="note2" placeholder="Note 2">
<input type="number" id="note3" placeholder="Note 3">
<button onclick="calculer()">Calculer</button>
<p id="resultat"></p>

<script>
function calculer() {
let n1 = parseFloat(document.getElementById("note1").value);
let n2 = parseFloat(document.getElementById("note2").value);
let n3 = parseFloat(document.getElementById("note3").value);
let moyenne = (n1 + n2 + n3) / 3;
document.getElementById("resultat").innerHTML =
"Moyenne : " + moyenne.toFixed(2);
}
</script>
</body>
</html>

```

5. Formulaire (15 min)

```

<form action="/envoyer" method="POST">
<label for="nom">Nom :</label>
<input type="text" id="nom" name="nom" required>

<label for="email">Email :</label>
<input type="email" id="email" name="email">

<label for="message">Message :</label>
<textarea id="message" name="message" rows="5"></textarea>

<button type="submit">Envoyer</button>
</form>

```

Types d'input : text, email, number, date, file, password, checkbox, radio.

6. Déploiement (10 min)

Options gratuites pour héberger un site enseignant :

- **GitHub Pages** : `username.github.io`
- **Netlify** : déploiement automatique depuis Git
- **Google Sites** : sans code (WYSIWYG)
- **Moodle** : espace cours interactif

7. Mini-projet (10 min)

Créer une page web personnelle pour votre futur métier d'enseignant :

1. Structure HTML (bio, cours, contact)
2. Style CSS (couleurs ENS : bleu foncé #003366)
3. Formulaire de contact
4. Lien vers le dépôt GitHub

QCM d'auto-évaluation

1. Que signifie HTML ?

a) HyperText Markup Language " b) High Tech Modern Language c) Home Tool Markup Language

2. Quelle balise HTML permet de créer un lien hypertexte ?

a) <link> b) <a> " c) <href>

3. Quel langage contrôle l'apparence d'une page Web ?

a) HTML b) CSS " c) JavaScript

4. Quel type d'input HTML est utilisé pour les mots de passe ?

a) text b) password " c) hidden

5. Qu'est-ce qu'une requête HTTP GET ?

a) Envoie des données au serveur via l'URL " b) Modifie la base de données c) Établit une connexion permanente

8. Conclusion (5 min)

HTML structure, CSS habille, JavaScript anime. Ces trois technologies sont la base du Web. Un enseignant moderne sait créer et publier du contenu en ligne.

Prochain cours : Git et collaboration.

Note pédagogique ENS

Transposition didactique : Le Web peut être enseigné au collège à travers la création de pages HTML simples. En SNT Seconde (thème "Web"), les élèves apprennent à distinguer client et serveur, à comprendre le rôle du navigateur, et à inspecter le code source d'une page. L'activité de mini-projet "page personnelle enseignant" peut être adaptée pour des élèves de 3^eme en leur faisant créer une page HTML sur un métier qui les intéresse. La séquence client-serveur peut être simulée par un jeu de rôle : un élève est le serveur, un autre le client.

Conseil pour l'enseignant formateur : Les étudiants ont souvent du mal avec la séparation HTML/CSS. Insistez sur le principe de responsabilité unique : HTML pour le fond, CSS pour la forme. Un exercice efficace consiste à fournir le même HTML et demander aux étudiants d'appliquer des CSS différents pour produire des rendus visuels totalement différents. Cela illustre la puissance et la flexibilité du CSS.

Prolongement : Les étudiants peuvent créer un mini-site complet d'enseignant avec plusieurs pages (accueil, cours, contact), un menu de navigation responsive (media queries), et un formulaire de contact fonctionnel utilisant un service comme Formspree ou un backend Flask simple. Ils peuvent également ajouter une page "portfolio de projets" avec des captures d'écran et des liens GitHub.

Templates : C:/Users/madan/Desktop/Supports de Cours 250626/Template/Pg-fr.docx | C:/Users/madan/Desktop/Supports de Cours 250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_2^eme_année_FR_07.pdf

Cours Informatique à l'ENS 2^{ème} année S^éance 07 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'ENS 2^{ème} année D^épartement de français

Durée : 1h30

Année universitaire : 2025-2026

Fiche cours à l'Plateforme e-learning (Moodle)

Résumé : benchmark 10 universités algériennes + pratiques USTO/Biskra/M'sila.

Public visé : Étudiants ENS 2^{ème} année D^épartement Français

Compétences pré-requises : Utilisation basique du terminal (ENS1 s^éance 03), projets Python

Objectifs transversaux :

- Appliquer la programmation orientée objet (classes, héritage, encapsulation) en Python
- Concevoir des algorithmes avancés (tri, recherche, récursivité) et estimer leur complexité
- Modéliser des données avec le modèle relationnel et requêtes SQL de base
- Comprendre l'architecture Web (HTML, CSS, JavaScript intro, HTTP)
- Utiliser Git et méthodes de travail collaboratif (branches, merge)
- Comparer l'enseignement de l'informatique au collège/lycée (Scratch, Python pédagogique)

Mode d'évaluation : TP POO + SQL 35 % à l'Projet intégré 40 % à l'Fiche s^éance pédagogique 25 %

Bibliographie : Chacon & Straub à l'*Pro Git* (git-scm.com) ; Atlassian à l'*Git Tutorials*

Objectifs pédagogiques spécifiques

À l'issue de cette s^éance, l'étudiant est capable de :

- Initialiser un dépôt Git
- Ajouter et commiter des fichiers
- Créer et fusionner des branches
- Push/pull sur GitHub
- Rédiger un README professionnel
- Utiliser .gitignore

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** à l'définit le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.

- **Programme national d'Informatique des ENS algériennes** fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Pourquoi Git ?
2. Installation et configuration
3. Commandes de base (init, add, commit, status)
4. Branches (branch, merge)
5. GitHub : remote, push, pull
6. README et documentation
7. Bonnes pratiques pour enseignants

1. Pourquoi Git ? (10 min)

Problème : "TD_final.docx", "TD_final2.docx", "TD_version_rendu.docx"...

Solution : Git système de contrôle de version.

Avantages :

- Historique complet des modifications
- Travail en équipe sans conflit
- Annulation d'erreurs possible
- Sauvegarde cloud (GitHub)
- Portfolio de code pour enseignant

2. Installation et configuration (10 min)

Installation

```
# Vérifier si Git est installé
git --version

# Télécharger : git-scm.com
```

Configuration initiale

```
git config --global user.name "Madani BELACEL"
git config --global user.email "m.belacel@ens-mostaganem.dz"
git config --global core.editor "code --wait"
```

3. Commandes de base (20 min)

```
# Initialiser un dépôt dans le dossier projet
cd projet-ens2
git init

# Voir l'état
git status

# Ajouter un fichier au suivi (stage)
git add cours_web.html
git add . # Ajouter tout le dossier

# Enregistrer une version (commit)
git commit -m "Ajout page web cours ENS2"

# Voir l'historique
git log --oneline
```

Cycle de vie d'un fichier :

```
flowchart LR
    A[Working Directory<br/>Fichiers modifiés] -->|git add| B[Staging Area<br/>Fichiers indexés]
    B -->|git commit| C[Repository<br/>Version sauvegardée]
    C -->|git checkout| A
    C -->|git push| D[Remote<br/>GitHub]
    D -->|git pull| C
    D -->|git clone| E[Autre machine]

    style A fill:#f9d71c,stroke:#333
    style B fill:#f9a825,stroke:#333
    style C fill:#4caf50,stroke:#333
    style D fill:#2196f3,stroke:#333
```

4. Branches (20 min)

Branche : version parallèle du projet. Permet de travailler sur une fonctionnalité sans affecter la branche principale.

```
# Lister les branches
git branch

# Créer une branche
git branch fonctionnalite-quiz

# Changer de branche
git checkout fonctionnalite-quiz

# Créer et basculer (en une commande)
git checkout -b nouvelle-fonctionnalite
```

Fusion (merge)

```
# Revenir sur main
git checkout main

# Fusionner la branche de travail
git merge fonctionnalite-quiz

# Supprimer la branche après fusion
git branch -d fonctionnalite-quiz
```

```
gitGraph
commit id: "Initial commit"
commit id: "Structure HTML"
branch fonctionnalite-quiz
checkout fonctionnalite-quiz
commit id: "Ajout quiz HTML"
commit id: "Style CSS quiz"
checkout main
commit id: "Correction navbar"
merge fonctionnalite-quiz id: "Merge quiz"
commit id: "README"
branch test-unitaire
checkout test-unitaire
commit id: "Test modele.py"
checkout main
merge test-unitaire id: "Merge tests"
commit id: "v1.0"
```

Conflits : quand deux branches modifient la même ligne. Git affiche les deux versions, à l'utilisateur de choisir.

5. GitHub (15 min)

Créer un dépôt distant

1. Créer un compte sur github.com
2. Cliquer sur **New repository**
3. Copier l'URL : `https://github.com/user/projet.git`

Lier le dépôt local à distant

```
git remote add origin https://github.com/user/projet.git
git push -u origin main
```

Mettre à jour

```
git push # Envoyer ses commits
git pull # Récupérer les commits des autres
```

GitHub Pages : Settings → Pages → branch: main → site web gratuit.

6. README et documentation (10 min)

Le fichier `README.md` est la page d'accueil du dépôt (Markdown).

```
# Projet ENS2 "Gestion de notes"

## Description
Application Web pour la gestion des notes des élèves.
Développée dans le cadre du module Informatique ENS 2ème année.

## Fonctionnalités
- Ajout/modification/suppression d'élèves
- Saisie de notes avec validation (0-20)
- Calcul automatique des moyennes
- Export CSV

## Technologies
- Python 3 + SQLite
- HTML/CSS/JavaScript
```

```
- Git + GitHub
```

```
## Installation
```

```
git clone https://github.com/user/projet-ens2.git
```

```
cd projet-ens2
```

```
python main.py
```

```
## Auteur
```

```
BELACEL Madani @ ENS Mostaganem
```

7. .gitignore (5 min)

Fichier `.gitignore` : ignore les fichiers qui ne doivent pas être versionnés.

```
__pycache__/  
*.pyc  
*.db  
.vscode/  
.env  
.DS_Store  
node_modules/
```

8. Bonnes pratiques (5 min)

1. Commits fréquents et atomiques (une modification = un commit)
2. Messages de commit clairs : `git commit -m "Ajout fonction calcul moyenne"`
3. Ne jamais commiter : mots de passe, clés API, données sensibles
4. Faire un `pull` avant de commencer à travailler
5. Utiliser des branches pour les expérimentations
6. Ajouter un README dès le début

QCM d'auto-évaluation

1. Quelle commande Git initialise un nouveau dépôt ?
a) `git start` b) `git init` c) `git new`
2. Quelle commande ajoute un fichier à la zone de staging ?
a) `git commit` b) `git add` c) `git push`
3. Une branche Git permet de :
a) Supprimer définitivement des fichiers b) Travailler en parallèle sans affecter la branche principale c) Accéder aux commits
4. Quelle commande envoie les commits vers GitHub ?
a) `git push` b) `git pull` c) `git clone`
5. Que contient le fichier `.gitignore` ?

a) La liste des contributeurs b) Les fichiers Ã ne pas versionner âœ c) Le message du dernier commit

9. Conclusion (5 min)

Git est l'outil standard du dÃ©veloppement moderne. Il permet de garder un historique, de collaborer et de prÃ©senter son travail (portfolio GitHub).

Prochain cours : Didactique de l'informatique â€” enseigner au collÃ©ge/lycÃ©e.

Note pÃ©dagogique ENS

Transposition didactique : Git peut Ãªtre introduit dÃ©s la classe de 3Ã¨me par une activitÃ© dÃ©branchÃ©e : chaque Ã©lÃ©ve reÃ§oit une feuille (fichier) et doit passer par des "commits" Ã chaque modification significative. Les branches sont reprÃ©sentÃ©es par des feuilles de couleur diffÃ©rente, et le merge par une agrafeuse qui relie les feuilles. En NSI PremiÃ¨re, le versionnement est au programme et peut Ãªtre prÃ©sentÃ© comme un carnet de bord du dÃ©veloppement. Le parallÃ¨le avec les "versions" d'un devoir rendu par un Ã©lÃ©ve est parlant.

Conseil pour l'enseignant formateur : Les Ã©tudiants ont souvent des difficultÃ©s avec les concepts de branches et de merge. Commencez par un scÃ©nario simple : "Vous travaillez sur le projet seul, vous crÃ©ez une branche pour une fonctionnalitÃ©, vous la fusionnez". Une fois ce cycle maÃ®trisÃ©, passez au travail collaboratif Ã deux sur le mÃªme dÃ©pÃ´t. Insistez sur l'importance des messages de commit clairs â€” c'est une compÃ©tence professionnelle.

Prolongement : Les Ã©tudiants peuvent crÃ©er un dÃ©pÃ´t Git de A Ã Z pour leur portfolio enseignant, en utilisant GitHub Pages pour le publier. Ils peuvent Ã©galement expÃ©rimer les conflits de fusion intentionnellement (deux Ã©tudiants modifient le mÃªme fichier) et apprendre Ã les rÃ©soudre â€” cette compÃ©tence est trÃ¨s valorisÃ©e dans le milieu professionnel.

Templates : C:/Users/madan/Desktop/Supports de Cours
250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours
250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_2Ã¨me_annÃ©e_FR_08.pdf

Cours Informatique à l'ENS 2^{ème} année à l'École Supérieure 08 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'ENS 2^{ème} année à l'École Supérieure de France

Durée : 1h30

Année universitaire : 2025-2026

Fiche cours à l'Plateforme e-learning (Moodle)

Résumé : benchmark 10 universités algériennes + pratiques USTO/Biskra/M'sila.

Public visé : Étudiants ENS 2^{ème} année à l'École Supérieure de France

Compétences pré-requis : Toutes les séances précédentes (synthèse du module)

Objectifs transversaux :

- Appliquer la programmation orientée objet (classes, héritage, encapsulation) en Python
- Concevoir des algorithmes avancés (tri, recherche, récursivité) et estimer leur complexité
- Modéliser des données avec le modèle relationnel et requêtes SQL de base
- Comprendre l'architecture Web (HTML, CSS, JavaScript intro, HTTP)
- Utiliser Git et méthodes de travail collaboratif (branches, merge)
- Comparer l'enseignement de l'informatique au collège/lycée (Scratch, Python pédagogique)

Mode d'évaluation : TP POO + SQL 35 % à l'Projet intégratif 40 % à l'Fiche séance pédagogique 25 %

Bibliographie : Eduscol à l'Programmes d'enseignement ; Scratch MIT ; Guide de l'enseignant à l'SNT

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Connaître les programmes officiels d'informatique au collège/lycée
- Concevoir une séance Scratch pour des élèves de collège
- Utiliser Python comme outil pédagogique
- Évaluer les compétences numériques (PIX/C2i)
- Rédiger une fiche de séance pédagogique

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** à l'définit le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.

- **Programme national d'Informatique des ENS algériennes** â€” fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.
- **Arrêté 712 fixant le programme des concours ENS** â€” détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algérien** â€” cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** â€” étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Programmes officiels (collège, lycée, SNT, NSI)
2. Scratch comme introduction
3. Python pédagogique
4. Évaluation des compétences numériques
5. Fiche de séance : module
6. Intelligence artificielle en classe
7. Charte d'usage du numérique

1. Programmes officiels (15 min)

Collège (Cycle 4)

- Algorithmique et programmation (Scratch)
- Internet et données
- Objets connectés
- **Volume** : ~15h en 3^{ème}

Lycée â€” SNT (Seconde)

- Sciences Numériques et Technologie
- Internet, Web, données, IA, géolocalisation
- **Volume** : 1h30/semaine

Lycée â€” NSI (Première/Terminale)

- Programmation Python (approfondie)
- Architecture, réseau, base de données
- **Volume** : 4h/semaine

```
mindmap
  root((Programmes<br/>Informatique))
    Collège Cycle 4
      Algorithmique Scratch
      Internet
      Données
      Objets connectés
    Volume: 15h 3ème
```

Lyc e SNT Seconde
 Internet & Web
 Donn es structur es
 Intelligence Artificielle
 G olocalisation
 Volume: 1h30/sem
 Lyc e NSI
 l re: Python, Web, Architecture
 Tale: R cursivit , BD, Projet
 Volume: 4h/sem
 Comp tences transversales
 PIX (France)
 C2i (Alg rie)
  thique num rique
 Charte d'usage

2. Scratch (20 min)

Scratch (scratch.mit.edu) : programmation visuelle par blocs   id al pour coll ge.

Concepts enseign s :

- Blocs de mouvement, apparence, son
- Boucles (r p ter, toujours)
- Conditions (si, si  alors)
- Variables et listes
-  v nements (quand drapeau cliqu , quand touche press e)

Exemple d'exercice : chat qui marche

1. Quand drapeau cliqu    avancer de 10 pas
2. Si touche espace press e   dire "Bonjour !" pendant 2 secondes
3. Rebondir si bord atteint

Progression recommand e :

1. Blocs simples (d placement)
2. Boucles et conditions
3. Variables (score, timer)
4. Projet final (jeu de pong, quiz)

```

flowchart LR
    subgraph College
        A[6e-5e<br/>D couverte Scratch] --> B[4e<br/>Blocs avanc s]
        B --> C[3e<br/>Introduction Python]
    end
    subgraph Lycee
        C --> D[2de SNT<br/>Python simple]
        D --> E[l re NSI<br/>Python approfondi]
        E --> F[Tale NSI<br/>Projets complexes]
    end
    subgraph ENS
        F --> G[ENS1<br/>Rappels Python]
        G --> H[ENS2<br/>POO, SQL, Web, Git]
    end
    style College fill:#e8f5e9,stroke:#2e7d32
  
```

```
style Lycee fill:#e3f2fd,stroke:#1565c0
style ENS fill:#fff3e0,stroke:#e65100
```

3. Python pédagogique (15 min)

Quand passer de Scratch à Python ?

- Scratch : 6e-4e (découverte, logique)
- Python : 3e-2de (première programmation textuelle)

Approche :

```
# Version Python d'un exercice Scratch
# Dire "Bonjour" quand on clique sur le drapeau

import turtle

t = turtle.Turtle()

# Avancer de 50 pas
t.forward(50)

# Tourner à droite
t.right(90)
t.forward(50)

turtle.done()
```

Bibliothèques pédagogiques :

- turtle : dessin, déplacement (transition Scratch)
- random : jeux, simulations
- tkinter : interfaces graphiques simples

4. Évaluation des compétences (10 min)

PIX (France)

Plateforme d'évaluation des compétences numériques (16 compétences).

- Moteur de recherche, email, fichiers, tableur, programmation
- Certification officielle

C2i (Algérie)

Certificat Informatique et Internet à niveau 1 (Licence).

Compétences évaluées :

- B1 : Recherche et veille informationnelle
- B2 : Identité numérique et RGPD
- B3 : Production de documents
- B4 : Collaboration à distance

5. Fiche de séance pédagogique (15 min)

Modèle pour concevoir une séance d'informatique au collège :

```
# Fiche de séance à€" Informatique Collège

**Niveau : 3ème
**Thème : Algorithmique et programmation
**Durée : 45 minutes

## Objectifs
- Comprendre la notion de boucle
- Utiliser une boucle "Répéter" en Scratch

## Prérequis
- Savoir déplacer un lutin
- Connaître les blocs "avancer" et "tourner"

## Déroulement
| Phase | Durée | Activité |
|-----|-----|-----|
| Découverte | 10 min | Présentation du carré magique |
| Recherche | 15 min | Réaliser un carré par répétition |
| Mise en commun | 10 min | Comparer les solutions |
| Synthèse | 5 min | Trace écrite : "Répéter 4 fois" |
| Évaluation | 5 min | Quiz Kahoot sur les boucles |

## Ressources
- Fiche à€lève
- Projet Scratch de départ
- Grille d'évaluation

## Bilan
- Points forts :
- Difficultés rencontrées :
- Remédiation :
```

6. IA en classe (10 min)

Opportunités :

- Assistant de correction (gagner du temps)
- Génération d'exercices personnalisés
- Traduction et adaptation
- Aide à la rédaction de fiches

Limites et précautions :

- Vérifier les résultats (IA peut se tromper)
- Ne pas saisir de données personnelles d'élèves
- Sensibiliser les élèves au plagiat
- Citer l'utilisation de l'IA

Règle d'or : l'IA est un outil, pas un substitut au jugement de l'enseignant.

7. Charte d'usage du numérique (5 min)

Il faut inclure dans une charte pour la classe :

1. Utiliser son compte institutionnel uniquement
2. Ne pas partager ses mots de passe
3. Respecter la vie privée des camarades
4. Signaler tout contenu inapproprié
5. Citer ses sources (images, textes)
6. Éteindre son téléphone en cours (sauf usage pédagogique)
7. Demander l'autorisation avant d'enregistrer/filmer

QCM d'auto-évaluation

1. À quel niveau Scratch est-il principalement enseigné ?
a) Lycée b) Collège (Cycle 4) c) Université
2. Quel est le volume horaire de l'option NSI en Première ?
a) 1h30/semaine b) 4h/semaine c) 6h/semaine
3. Quelle bibliothèque Python permet une transition pédagogique depuis Scratch ?
a) numpy b) turtle c) django
4. PIX est une plateforme d'évaluation :
a) Des compétences en programmation uniquement b) Des compétences numériques globales c) Des langues étrangères
5. Quelle est la règle d'or concernant l'IA en classe ?
a) L'IA remplace l'enseignant b) L'IA est un outil, pas un substitut au jugement de l'enseignant c) L'IA est interdite dans l'éducation

8. Conclusion (5 min)

En tant que futur enseignant, vous devez maîtriser à la fois les concepts informatiques et leur didactique. Scratch pour le collège, Python pour le lycée, et une éthique numérique solide pour tous.

Prochain cours : Conception du projet ENS2 à l'application Web ou module Python POO.

Note pédagogique ENS

Transposition didactique : Cette séance est elle-même un exemple de transposition didactique : les concepts informatiques avancés (POO, SQL, Web) sont "transposés" en activités accessibles pour le secondaire. La progression Scratch à Python (section 3) est au cœur des programmes actuels. Le modèle de fiche de séance proposé peut être utilisé pour n'importe quel niveau. Les étudiants-enseignants doivent comprendre que la difficulté n'est pas de maîtriser l'informatique, mais de l'enseigner à des débutants c'est tout l'enjeu de la didactique.

Conseil pour l'enseignant formateur : Organisez un micro-enseignement : chaque étudiant prépare et anime une séquence de 10 minutes sur un concept informatique (boucle, condition, variable) devant le groupe. Filmez les prestations pour un retour constructif. L'évaluation par les pairs (grille critique) renforce la professionnalisation. N'oubliez pas de sensibiliser à la gestion de classe

: que faire quand un élève bloque sur un exercice ? Comment différencier ?

Prolongement : Les étudiants peuvent concevoir une séquence complète de 3 séances sur un thème informatique (création d'un jeu Scratch, initiation à Python avec Turtle, découverte du Web avec HTML) incluant fiches élèves, grilles d'évaluation, et traces écrites. Ces productions seront réinvesties pendant le stage pratique.

Templates : C:/Users/madan/Desktop/Supports de Cours
250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours
250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_2ème_année_FR_09.pdf

Cours Informatique à l'ENS 2^{ème} année à l'École Supérieure 09 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'ENS 2^{ème} année à l'École Supérieure de France

Durée : 1h30

Année universitaire : 2025-2026

Fiche cours à l'Plateforme e-learning (Moodle)

Résumé : benchmark 10 universités algériennes + pratiques USTO/Biskra/M'sila.

Public visé : Étudiants ENS 2^{ème} année à l'École Supérieure de France

Compétences pré-requises : Toutes les séances du module (POO, SQL, Web, Git, didactique)

Objectifs transversaux :

- Appliquer la programmation orientée objet (classes, héritage, encapsulation) en Python
- Concevoir des algorithmes avancés (tri, recherche, récursivité) et estimer leur complexité
- Modéliser des données avec le modèle relationnel et requêtes SQL de base
- Comprendre l'architecture Web (HTML, CSS, JavaScript intro, HTTP)
- Utiliser Git et méthodes de travail collaboratif (branches, merge)
- Comparer l'enseignement de l'informatique au collège/lycée (Scratch, Python pédagogique)

Mode d'évaluation : TP POO + SQL 35 % à l'Projet intégré 40 % à l'Fiche séance pédagogique 25 %

Bibliographie : Documentation Python, Flask, SQLite, GitHub

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Planifier un projet logiciel complet
- Concevoir l'architecture d'une application Web
- Structurer le code en modules séparés
- Utiliser Flask pour créer une application Web simple
- Appliquer Git tout au long du développement

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** à l' définit le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algériennes** à l' fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.

- **Arrêté 712 fixant le programme des concours ENS** : détermine les épreuves et les programmes des concours d'accès aux ENS, en cohérence avec les contenus enseignés.
- **Référentiel des compétences numériques algébriques** : cadre national (en cours d'élaboration) définissant les compétences numériques attendues des enseignants.
- **Benchmark des 10 universités algériennes** : étude comparative des pratiques d'enseignement de l'informatique, consultée pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Cahier des charges du projet ENS2
2. Architecture de l'application
3. Choix techniques
4. Structure des fichiers
5. Développement guidé : squelette Flask
6. Connexion base de données
7. Planification avec GitHub Projects

1. Cahier des charges (10 min)

Projet ENS2 : Application de gestion de notes pour enseignant

Fonctionnalités obligatoires

- Ajouter/modifier/supprimer des élèves
- Ajouter des notes avec validation (0-20)
- Calcul automatique des moyennes
- Affichage d'un relevé de notes (tableau)
- Export CSV

Fonctionnalités optionnelles

- Graphiques (matplotlib)
- Authentification (login/mot de passe)
- Thème clair/sombre
- Mode multilingue (FR/EN/AR)

Contraintes techniques

- Python 3 + SQLite
- Flask (Web) ou interface console + export HTML
- Git + GitHub (dépôt public)
- README.md complet
- Fiche séance pédagogique associée

2. Architecture (10 min)

```

projet-ens2/
├── app.py # Point d'entr e Flask
├── modele.py # Classes POO (Eleve, Note, Matiere)
├── database.py # Connexion et requ tes SQLite
├── templates/ # Templates HTML (Jinja2)
│   ├── index.html
│   ├── eleves.html
│   └── releve.html
├── static/ # CSS, JS, images
│   ├── style.css
│   └── script.js
├── donnees/ # Fichiers de donn es
│   └── ecole.db
├── tests/ # Tests unitaires
│   └── test_modele.py
├── requirements.txt # D pendances
├── .gitignore
└── README.md

```

```

graph TD
    subgraph Vue [Vue - Templates HTML]
        A[index.html] --> B[eleves.html]
        A --> C[releve.html]
    end
    subgraph Controleur [Contr leur - Flask]
        D[app.py] --> E[Routes]
        E --> F[/]
        E --> G[/eleves]
        E --> H[/ajouter]
        E --> I[/releve]
    end
    subgraph Modele [Mod le - Python + SQLite]
        J[modele.py] --> K[Classes POO]
        K --> L[Eleve]
        K --> M[Note]
        L --> N[database.py]
        M --> N
        N --> O[ecole.db]
    end
    D --> A
    D --> B
    D --> C
    J --> N

    style Vue fill:#e3f2fd,stroke:#1565c0
    style Controleur fill:#fff3e0,stroke:#e65100
    style Modele fill:#e8f5e9,stroke:#2e7d32

```

Principe : s paration des responsabilit s (mod le, vue, contr leur   MVC).

3. Choix techniques (10 min)

Composant	Technologie	Justification
Langage	Python 3	Simple, tout-en-un
Framework Web	Flask	L�ger, facile � apprendre
Base de donn�es	SQLite	Fichier unique, pas de serveur
Templates	Jinja2 (int�gr� Flask)	Syntaxe simple

CSS	Bootstrap 5	Rapide, responsive
Versioning	Git + GitHub	Portfolio, collaboration
D��ploiement	GitHub Pages ou PythonAnywhere	Gratuit

4. Code guid   Flask (20 min)

```
# app.py
from flask import Flask, render_template, request, redirect
from modele import Eleve, Note
from database import init_db, ajouter_eleve, get_eleves

app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/eleves")
def liste_eleves():
    eleves = get_eleves()
    return render_template("eleves.html", eleves=eleves)

@app.route("/ajouter", methods=["POST"])
def ajouter():
    nom = request.form["nom"]
    prenom = request.form["prenom"]
    groupe = request.form["groupe"]
    ajouter_eleve(nom, prenom, groupe)
    return redirect("/eleves")

if __name__ == "__main__":
    init_db()
    app.run(debug=True)
```

5. Mod  le POO (15 min)

```
# modele.py
class Eleve:
    def __init__(self, id, nom, prenom, groupe):
        self.id = id
        self.nom = nom
        self.prenom = prenom
        self.groupe = groupe
        self.notes = []

    def moyenne(self):
        if not self.notes:
            return 0.0
        return sum(n.valeur for n in self.notes) / len(self.notes)

    def ajouter_note(self, matiere, valeur):
        if 0 <= valeur <= 20:
            self.notes.append(Note(self.id, matiere, valeur))

class Note:
    def __init__(self, eleve_id, matiere, valeur):
        self.eleve_id = eleve_id
        self.matiere = matiere
        self.valeur = valeur
```

6. Base de données (10 min)

```
# database.py
import sqlite3

DB = "donnees/ecole.db"

def init_db():
    conn = sqlite3.connect(DB)
    c = conn.cursor()
    c.execute("""
CREATE TABLE IF NOT EXISTS eleves (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nom TEXT NOT NULL,
    prenom TEXT NOT NULL,
    groupe TEXT
)
""")
    c.execute("""
CREATE TABLE IF NOT EXISTS notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    eleve_id INTEGER NOT NULL,
    matiere TEXT NOT NULL,
    valeur REAL NOT NULL CHECK(valeur >= 0 AND valeur <= 20),
    FOREIGN KEY(eleve_id) REFERENCES eleves(id)
)
""")
    conn.commit()
    conn.close()

def get_eleves():
    conn = sqlite3.connect(DB)
    conn.row_factory = sqlite3.Row
    c = conn.cursor()
    c.execute("SELECT * FROM eleves")
    eleves = [dict(row) for row in c.fetchall()]
    conn.close()
    return eleves
```

7. Planification (5 min)

Utiliser GitHub Projects ou un tableau Kanban :

Sprint	Date limite	Livrable
Sprint 1	J+7	Base de données + modèle POO
Sprint 2	J+14	Interface Flask (HTML/CSS)
Sprint 3	J+21	Fonctionnalités complètes
Sprint 4	J+28	Tests + Déploiement
Sprint 5	J+30	Soutenance

```
gantt
title Planning Projet ENS2
dateFormat YYYY-MM-DD
axisFormat %d/%m

section Sprint 1
```

Modèle POO + Base SQLite :done, s1, 2026-02-01, 7d

section Sprint 2

Interface Flask + Templates :done, s2, after s1, 7d

section Sprint 3

Fonctionnalités CRUD :active, s3, after s2, 7d

section Sprint 4

Tests + Déploiement :s4, after s3, 7d

section Sprint 5

Soutenance :milestone, after s4, 1d

QCM d'auto-évaluation

1. Quel framework Web est utilisé dans le projet ENS2 ?

a) Django b) Flask c) FastAPI

2. Quel est le système de gestion de base de données utilisé ?

a) MySQL b) PostgreSQL c) SQLite

3. Quel modèle d'architecture est suivi par le projet ?

a) MVC (Modèle-Vue-Contrôleur) b) MVP c) Microservices

4. Quelle fonctionnalité est obligatoire dans le projet ?

a) Authentification b) Export CSV c) Mode multilingue

5. Combien de sprints sont prévus dans la planification ?

a) 3 b) 5 c) 7

8. Conclusion (5 min)

Le projet ENS2 est l'aboutissement du module. Il mobilise toutes les compétences acquises : POO, bases de données, Web, Git, et didactique.

Prochain cours : Soutenance du projet ENS2 à présentation et démonstration.

Note pédagogique ENS

Transposition didactique : La méthodologie de projet présentée ici (cahier des charges, architecture, sprints) peut être transposée au collège/lycée sous forme de projet Scratch ou Python. L'important est de suivre la même démarche : 1) définir le besoin, 2) concevoir la solution, 3) implémenter par itérations, 4) tester, 5) présenter. En NSI Terminale, le projet est une épreuve du baccalauréat à cette séance prépare donc directement les futurs enseignants à accompagner leurs élèves.

Conseil pour l'enseignant formateur : Le projet ENS2 est le moment de valoir du module. Veillez à ce que chaque étudiant ait un projet fonctionnel minimal avant de viser des fonctionnalités avancées ("Make it work, then make it better"). Encouragez la collaboration mais valorifiez les contributions individuelles via Git (chaque étudiant doit commiter). Prévoyez un moment de revue de code entre pairs avant la soutenance.

Prolongement : Les Étudiants peuvent Étendre le projet avec des fonctionnalités avancées : import CSV (pour charger une classe existante), génération de graphiques de répartition des notes, envoi de relevés de notes par email, ou interface d'administration avec login. Ces extensions peuvent être présentées comme "bonus" lors de la soutenance et valorisées dans la note.

Templates : C:/Users/madan/Desktop/Supports de Cours
250626/Template/PG-fr.docx | C:/Users/madan/Desktop/Supports de Cours
250626/Template/Presentation_BELACEL_V3.pptx

PDF : Cours_Informatique_ENS_2^{ème}_année_FR_10.pdf

Cours Informatique à l'ENS 2^{ème} année à l'École Supérieure 10 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS à l'ENS 2^{ème} année à l'École Supérieure de France

Durée : 1h30

Année universitaire : 2025-2026

Fiche cours à l'Plateforme e-learning (Moodle)

Résumé : benchmark 10 universités algériennes + pratiques USTO/Biskra/M'sila.

Public visé : Étudiants ENS 2^{ème} année à l'École Supérieure de France

Compétences pré-requis : Projet ENS2 développé (séance 09), Git, POO, SQL, Flask

Objectifs transversaux :

- Appliquer la programmation orientée objet (classes, héritage, encapsulation) en Python
- Concevoir des algorithmes avancés (tri, recherche, récursivité) et estimer leur complexité
- Modéliser des données avec le modèle relationnel et requêtes SQL de base
- Comprendre l'architecture Web (HTML, CSS, JavaScript intro, HTTP)
- Utiliser Git et méthodes de travail collaboratif (branches, merge)
- Comparer l'enseignement de l'informatique au collège/lycée (Scratch, Python pédagogique)

Mode d'évaluation : TP POO + SQL 35 % à l'Projet intégré 40 % à l'Fiche séance pédagogique 25 %

Bibliographie : MDN Web Docs ; Flask Documentation ; Git Book

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Présenter un projet logiciel complet
- Démontrer les fonctionnalités développées
- Expliquer les choix techniques et d'architecture
- Montrer le dépôt Git avec historique
- Présenter la fiche séance pédagogique associée

Contexte réglementaire algérien

Ce cours s'inscrit dans le cadre réglementaire algérien de formation des enseignants :

- **Arrêté ministériel portant création des ENS (cycle Licence/Master)** à l' définit le cadre institutionnel des Écoles Normales Supérieures et leur mission de formation des enseignants.
- **Programme national d'Informatique des ENS algériennes** à l' fixe les objectifs, contenus et volumes horaires du module Informatique dans la formation initiale des enseignants.

- **Arrêté 712 fixant le programme des concours ENS** â€” d termine les  preuves et les programmes des concours d'acc s aux ENS, en coh rence avec les contenus enseign s.
- **R f rentiel des comp tences num riques alg rien** â€” cadre national (en cours d' laboration) d finissant les comp tences num riques attendues des enseignants.
- **Benchmark des 10 universit s alg riennes** â€”  tude comparative des pratiques d'enseignement de l'informatique, consult e pour aligner ce cours sur les standards nationaux.

Plan du cours

1. Crit res d' valuation d taill s
2. Structure de la pr sentation (10 min)
3. D mo en direct
4. Questions techniques
5. Fiche s ance p dagogique
6. Bilan du module ENS2
7. Prochaines  tapes (stage, portfolio)

1. Crit res d' valuation (5 min)

Crit�re	Poids	D�tail
Fonctionnalit�s	30 %	CRUD �l�ves/notes, moyenne, export CSV
Code POO	20 %	Classes, h�ritage, m�thodes
Base de donn�es	15 %	SQLite, tables, requ�tes, jointures
Interface Web	10 %	Flask, HTML/CSS, templates
Git	10 %	Commits r�guliers, branches, README
Fiche p�dagogique	10 %	S�ance coll�ge/lyc�e avec Scratch
Pr�sentation orale	5 %	Clart�, structure, d�mo

2. Structure de la pr sentation (10 min)

Chaque  tudiant dispose de 10 minutes :

```
mindmap
  root((Projet ENS2<br/>Soutenance 10 min))
    Pr sentation
    Objectif
    Public cible
    Contexte scolaire
    Architecture
    Sch ma MVC
    Technologies
    Base de donn es
    D mo en direct
    Ajout  l ve
    Saisie notes
    Relev  de notes
```

Export CSV
 Code
 Classe Eleve POO
 Requêtes SQL
 Routes Flask
 Templates HTML
 Versioning
 Git historique
 Branches
 README
 .gitignore
 Pédagogie
 Fiche séance collège
 Progression Scratch
 Évaluation
 Conclusion
 Bilan compétences
 Perspectives
 Questions

Minute	Contenu
0-1	Présentation du projet (objectif, public cible)
1-3	Architecture (schéma, technologies)
3-5	Démonstration en direct (ajout élève, saisie notes, relevé)
5-7	Code : classe Eleve, requête SQL, route Flask
7-8	Git : historique, branches, README
8-9	Fiche séance pédagogique associée
9-10	Conclusion + questions

3. Démonstration en direct – scénario type (20 min)

Scénario de démonstration :

- Accueil** – Page d'accueil avec présentation
- Ajout d'élève** – Saisir "Belacel", "Ali", "ENS2-A"
- Ajout de notes** – Informatique: 15, Français: 12, Anglais: 18
- Calcul de moyenne** – Vérifier que la moyenne s'affiche correctement
- Relevé de notes** – Afficher le tableau complet
- Export CSV** – Télécharger le fichier et l'ouvrir dans Excel
- GitHub** – Montrer le dépôt, les commits, le README

Erreurs à anticiper :

- Ajouter une note > 20 – message d'erreur attendu
- Supprimer un élève – confirmation
- Navigation – responsive (mobile/desktop)

4. Questions techniques possibles (15 min)

Préparer les réponses à ces questions :

POO :

- Pourquoi as-tu choisi l'héritage dans ton modèle ?
- Quelle est la différence entre attribut de classe et d'instance ?
- Comment as-tu implémenté l'encapsulation ?

Base de données :

- Pourquoi SQLite plutôt que MySQL/PostgreSQL ?
- Quelle est la clé primaire ? Pourquoi AUTOINCREMENT ?
- Comment as-tu géré la contrainte 0 ≤ note ≤ 20 ?

Web :

- Explique le cycle requête/réponse dans Flask
- Comment passes-tu des données du Python au HTML ?
- Qu'est-ce qu'une route dans Flask ?

Git :

- Montre un commit où tu as corrigé un bug
- Quelle est la différence entre merge et rebase ?
- Comment gères-tu un conflit de fusion ?

Pédagogie :

- Comment adapterais-tu ce projet pour des élèves de 3^{ème} ?
- Quelle trace écrite proposes-tu aux élèves ?
- Comment évalues-tu les compétences PIX ?

5. Fiche séance pédagogique (15 min à€" travail individuel)

Rédiger une fiche de séance de 45 minutes pour le collège, en lien avec le projet.

Sujet proposé : Découvrir la programmation avec Scratch à€" Créer un quiz interactif

Utiliser le modèle vu à€" la séance 08 :

```
# Fiche de séance à€" Quiz interactif avec Scratch

**Niveau : 3ème (Cycle 4)
**Thème : Algorithmique et programmation
**Durée : 45 minutes

## Objectifs
- Utiliser une variable (score)
- Implémenter une condition (si la réponse est correcte)
- Créer une boucle de jeu

## Déroulement
1. Découverte (5 min) : montrer le quiz finalisé
2. Variables (10 min) : créer variable "score"
3. Questions (15 min) : ajouter 3 questions avec conditions
4. Test (10 min) : jouer et debugger
5. Bilan (5 min) : trace écrite
```

```
## Ãvaluation
- Quiz final fonctionnel (3 questions minimum)
- Variable score qui s'incrÃmente
- Message de feedback (bravo/essaye encore)
```

6. Bilan du module ENS2 (10 min)

```
quadrantChart
title CompÃtences acquises â€” Module Informatique ENS2
x-axis "Technique" --> "PÃdagogique"
y-axis "Base" --> "AvancÃ"
quadrant-1 "Expertise pÃdagogique"
quadrant-2 "Expertise technique"
quadrant-3 "Fondamentaux"
quadrant-4 "En dÃveloppement"
POO Python: [0.75, 0.85]
Algorithmique: [0.80, 0.70]
Bases de donnÃes: [0.70, 0.75]
DÃveloppement Web: [0.65, 0.60]
Git: [0.60, 0.55]
Didactique: [0.30, 0.80]
Projet intÃgrÃ:
```

CompÃtences acquises :

SÃance	CompÃtence	Niveau atteint
01	Architecture du module	MaÃtrise
02-03	POO Python (classes, hÃritage, encapsulation)	MaÃtrise
04	Algorithmes avancÃs (rÃcursivitÃ, tris)	Approfondi
05	Bases de donnÃes et SQL	MaÃtrise
06	DÃveloppement Web (HTML/CSS/JS)	Approfondi
07	Git et collaboration	MaÃtrise
08	Didactique de l'informatique	Sensibilisation
09-10	Projet intÃgrÃ	SynthÃse

Objectifs atteints :

- â€œ... Programme orientÃ objet en Python
- â€œ... RequÃtes SQL et base de donnÃes
- â€œ... Application Web fonctionnelle
- â€œ... DÃpÃt Git avec historique
- â€œ... Fiche de sÃance pÃdagogique
- â€œ... PrÃsentation et dÃmonstration

7. Prochaines Ãtapes (5 min)

Stage pratique :

- Stage d'observation en Ãtablissement scolaire (2 semaines)

- Mettre en pratique la didactique de l'informatique
- Tester les fiches de séance avec des élèves

Portfolio numÃ©rique :

- Page GitHub Pages présentant vos projets
- Lien vers le dépôt du projet ENS2
- CV et lettres de motivation

Certifications :

- PIX (certification des compétences numériques)
- C2i (Certificat Informatique et Internet)
- TOEFL/IELTS (si prévu)

8. Conclusion (5 min)

Félicitations ! Le module Informatique ENS vous a donné les bases solides pour enseigner le numérique au 21^{ème} siècle. Vous repartez avec :

[illegible]

L'informatique n'est pas qu'une mati re   c'est une comp tence transversale pour tous les enseignants.

QCM d'auto-évaluation

1. Quelle est la durÃ©e de la prÃ©sentation orale du projet ?

a) 5 minutes b) 10 minutes c) 20 minutes

2. Quel est le poids du code POO dans l'Évaluation ?

a) 10 % b) 20 % c) 30 %

3. Quelle erreur faut-il anticiper lors de la dÃ©mo ?

a) Ajouter une note > 20 → message d'erreur → b) Le serveur qui explose c) La base de données qui se supprime

4. Quel est le but de la fiche sÃ©ance pÃ©dagogique dans le projet ?

a) Décrire l'architecture du code b) Montrer comment enseigner le projet des élèves de collège/lycée c) Lister les bugs connus

5. Apr s le module ENS2, quelle certification num rique est recommand e ?

a) TOEFL b) PIX âœ“ c) CCNA

Conclusion

Félicitations ! Vous avez complété le module Informatique ENS 2^{ème} année. Vous repartez avec des compétences solides en programmation orientée objet, algorithmique, bases de données, développement Web, versionnement Git et didactique de l'informatique. Ces outils vous accompagnent dans votre carrière d'enseignant et dans la préparation de vos futurs séjours aux défis numériques.

Prochaines étapes : Stage pratique en établissement, construction du portfolio enseignant, et certifications PIX/C2i.

Note pédagogique ENS

Transposition didactique : Cette séance de soutenance est elle-même un modèle pour les enseignants : elle montre comment structurer une évaluation de projet (critères transparents, démo en direct, questions techniques, fiche pédagogique). Ce format peut être adapté pour des séjours de collège (soutenance de projet Scratch de 5 minutes) ou de lycée (soutenance NSI de 15-20 minutes). L'évaluation par grille critériée est une pratique recommandée dans le programme officiel.

Conseil pour l'enseignant formateur : La soutenance est un moment clé : elle évalue non seulement le code mais aussi la capacité à communiquer et à argumenter. Encouragez les étudiants à préparer un script et à répéter. Prévoyez un vidéoprojecteur pour la démo et testez-le avant. Les questions techniques (section 4) doivent être adaptées au niveau de chaque étudiant. Un étudiant plus faible peut se voir poser des questions sur les bases (différence classe/instance) tandis qu'un étudiant avancé peut être interrogé sur les choix d'architecture.

Prolongement : Le projet ENS2 devient la pièce maîtresse du portfolio enseignant. Les étudiants sont encouragés à :

- Présenter le projet lors d'entretiens pour des postes d'enseignant
- L'améliorer pendant le stage pratique avec des retours d'élèves
- Le publier sur GitHub Pages comme démonstration de leurs compétences
- L'étendre avec des fonctionnalités inspirées de leur expérience de stage